

Lift Off!

Visual Servoing of Space Shuttle Model

Issac Ross

05/18/2018

Abstract

This report documents a year-long project to autonomously control a model space shuttle. This model operates on an air table with fans acting as thrusters, and the automation is accomplished through feedback control which uses computer vision as the sensor. The structure of this report is meant to provide a reproducible procedure for augmenting existing models, setting up the computer vision system, and establishing wireless communication, as well as explain the approach and inner workings of the code. In the proceeding report, I will outline the project at a high level, then discuss my analysis of each major physical and electrical component of the system, followed by in-depth descriptions of the computer vision detection, controls used for auto-piloting the shuttle, finishing with discussion of the overall outcome.

Contents

1	Introduction and System Overview	3
2	Setup	5
2.1	Bill of Materials	5
2.2	PC Software	6
2.3	Raspberry Pi Setup	6
2.4	Arduino Board Setup	6
2.5	Xbee Configuration - X-CTU	7
3	Xbee Communication	8
4	Joystick Controller	9
4.1	Electrical Schematic	9
4.2	Commands and Control	11
5	Shuttle System	12
5.1	Electrical Schematic	13
5.2	Battery Improvements	13
5.3	Physical System ID	15
5.3.1	Thrusters (Fans)	15
5.3.2	Physical Model	17
5.4	Receiving & Interpreting Wireless Commands	18
6	Computer Vision	21
6.1	Camera and Image Settings	21
6.2	Object Detection	23
6.2.1	Thresholding and Object Refinement	23
6.2.2	Position and Orientation Detection	24
6.3	Alternative Tracking Method	27
6.4	Augmented Reality Overlay	29
6.4.1	Booster Flames	29
6.4.2	Projected Shuttle	31
6.4.3	Shuttle Trail	32
7	Control	33
7.1	Control Command	33
7.2	Controller Design	35
7.2.1	Simulation of Physical Model	35
7.2.2	Discretization of Controllers	39
7.3	Implementation	40
8	Testing	42
8.1	Test Procedure	42
8.2	Data Collection	43
8.3	Results Discussion	43
9	Conclusion	44

A Joystick Controller Arduino Uno	45
B Shuttle Arduino Mega	46
C Python Computer Vision	51
C.1 Main Loop	51
C.2 Setup	51
C.3 Target Detection	54
C.4 Image Reading	54
C.5 Image Processing	55
C.6 Sobel Detection	55
C.7 FindContour() Detection	56
C.8 Position Calculator	56
C.9 Command Calculator	57
C.10 Flame Projection	58
C.11 Position Overlay	60
C.12 Save Data	60
D Data Processing	61

1 Introduction and System Overview

This report is a living document meant to comprehensively explain the state of the Computer Vision Detection and Remote Control of Space Shuttle Model project. Kyle Samples produced the hardware and a functional form of the shuttle model and controller, which I have supplemented and improved upon. The major capacity in which the project was altered is the use of Python and OpenCV for computer vision detection, as opposed to the Microsoft Kinect previously used. Additionally, the automation controls were improved to allow for complete integration within the shuttle docking exhibit at the Denver Museum of Nature and Science eventually.

Before diving fully into details, an overview of the system will be helpful in understanding how the functional connections discussed later on work. A graphical representation of the system is shown on the following page in Fig. 2. The main focus of this project is the shuttle and the Raspberry Pi conducting computer vision detection. The shuttle takes commands from both the joystick controller and the Pi in the form of serial communications sent over Xbee wireless devices. The joystick controller enables a user to pilot the shuttle manually, or activate auto-pilot, which is handled by the Pi and shuttle's MCU. To conduct auto-piloting, the Pi uses images captured by a camera to detect the shuttle and passes that data to the shuttle itself, which controls the attached fans to move it and correct it's position. The details of how these commands are formatted, sent, interpreted, translated into action will be detailed on a system by system level over the next few sections, starting with the Joystick Controller.

Throughout this report I will make mention of the components of the system as portions of the standard feedback controller. That system is shown in Fig. 1, and will be elaborated on throughout the report.

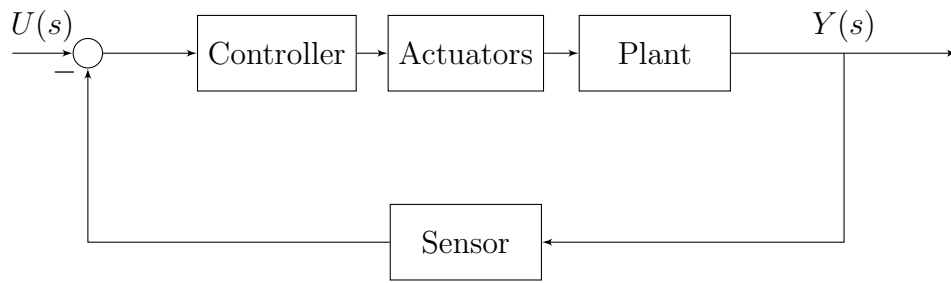


Figure 1: Feedback Control Diagram of System

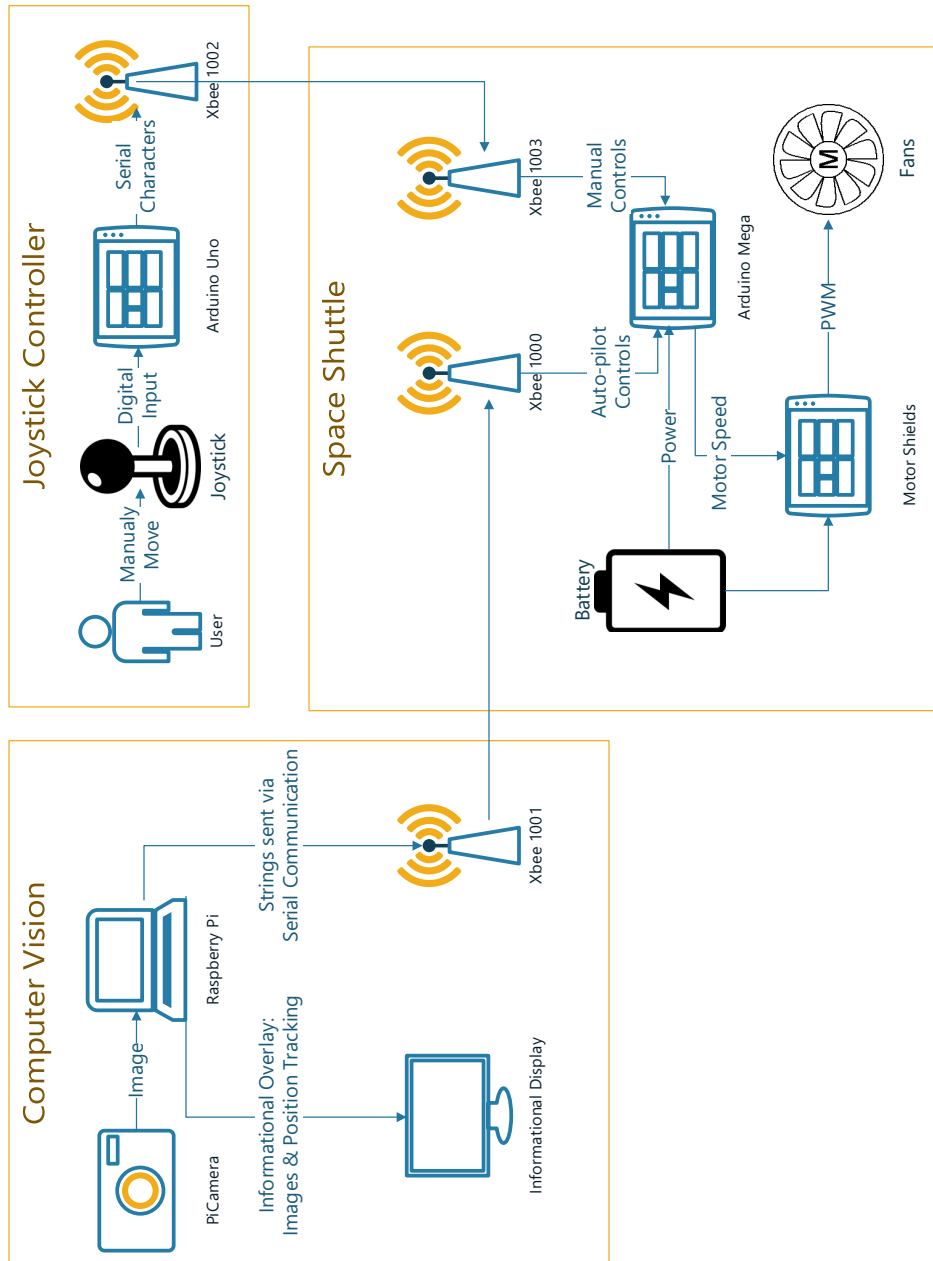


Figure 2: System Overview

2 Setup

In this section, I will discuss all the necessary hardware, software, code, and procedures for setting up the system. The code will not be discussed in detail until the further sections of this report. Firstly, before any software can be loaded, the physical hardware and microcontrollers must be acquired and assembled, according to the CAD of the system included in the TDP and discussed in section 5.3.2, and the wiring diagrams contained in sections 4.1 and 5.1.

2.1 Bill of Materials

Table 1 shows a bill of materials for the system, along with cost or estimations of cost for each component. The prices are subject to change as some components may be replaced over the course of the project, but this gives a good ballpark for the cost of the system.

Table 1: Bill of Materials

Item	Quantity	Cost (Estimation)
Joystick	2	(\$25)
Slide Switch	2	\$1
Toggle Switch	1	\$5
Acrylic	150 in ²	(\$10)
Breadboard	2	\$5
Arduino Uno	1	\$25
Arduino Mega	1	\$40
Arduino Motor Shield	2	\$9
Xbee Shield	3	\$15
Xbee Explorer USB Adapter	1	\$25
Case Fans	6	\$10
Battery (9v)	1	\$3
LiPo Battery (11.1v, 3000 mAh)	1	\$26
Model Structural Material	250 in ²	(\$20)
Raspberry Pi	1	\$40
PiCamera	1	\$10
PiCamera Ribbon Cable (2m)	2	\$6
PiCamera Ribbon Cable Extender	1	\$3
Total		\$363

2.2 PC Software

In order to interact with the hardware discussed above, a few programs are needed. This list includes some programs which may only run on a Windows PC, and does not include the software required for CV on the Raspberry Pi. Firstly, in order to program the Arduino boards, the Arduino IDE is necessary. This may be found at <https://www.arduino.cc/en/Main/Software>. Next, X-CTU, which will be discussed in detail shortly, must be installed on a machine for the purpose of programming and testing Xbees. This software may be found at <https://www.digi.com/products/xbee-rf-solutions/xctu-software/xctu>. Solidworks may be necessary to view the CAD I have created of the project, which can be downloaded from CU Boulder's ITLL website, or from D'assault Systemes' website.

2.3 Raspberry Pi Setup

Apart from the PC used to create Arduino code and test Xbees, the Raspberry Pi is used to conduct the computer vision detection in this project. The lightweight and inexpensive system makes for an ideal computer to be installed in a display. The recommended starting package for this system is NOOBS, which comes with the languages and IDEs we need, allowing for incredibly simple setup. This may be purchased on an SD card, or downloaded onto any SD card over 4GB in memory. This OS and software package includes the IDLE Python IDE which serves our purposes, especially if someone is simply reproducing this project. The version of Python used for this project is Python 2.7, which will conflict with any 3.0 or higher versions in a few syntactical ways, so be sure to use 2.7.

Some Python libraries are not included on a fresh install and must be installed manually. To do so, open the command prompt and enter the command "sudo apt-get install python-(*package name*)". The packages used for this project are: picamera, cv2, math, numpy, time, and serial, some of which may come with NOOBS. With these packages installed, the code provided in the TDP, and appendix C should run without any issue or additional manipulation necessary.

2.4 Arduino Board Setup

With the PC upon which the Arduino IDE was installed, the code for the joystick and shuttle must be uploaded to the Arduino Uno and Mega respectively. Each of these boards use the standard USB cable to upload code, though each must be uploaded to using the correct setting for board type within the IDE's settings. The code for each board may be found in the TDP, or in appendices A and B. Testing of the shuttle code may be conducted with the USB cable still attached and serial monitor active to receive messages from the board about its state.

2.5 Xbee Configuration - X-CTU

When using Xbees, our wireless communication method, they must be configured to know what channel to send data on, and where to send it to. To do so, I use X-CTU, a software produced by Digi, who also makes the Xbees. This software has two valuable features. Firstly, it allows simple reading and writing of configurations of Xbee devices, and secondly, it can display information received by a connected Xbee. The Xbee is connected using an Explorer shield, which facilitates a USB to serial pin connection. In our case, there are four Xbees, forming two pairs. In each pair, data is sent from one Xbee to it's 'target' and received by another whose 'address' matches. This network configuration is accomplished by the fields shown in Fig. 3. Additionally, the serial settings must be the same between the different Xbees and their respective device. For instance, all our Xbees use a Baud Rate (data interface speed) of 115200, and would not properly communicate with the Arduino if it were looking for one running at a lower speed. These settings are shown in Fig. 4.

CH Channel	C	 
ID PAN ID	1337	 
DH Destination Address High	0	 
DL Destination Address Low	1000	 
MY 16-bit Source Address	1001	 

Figure 3: Network Configuration Settings of Xbee

BD Interface Data Rate	115200 [7]	 
NB Parity	No Parity [0]	 
RO Packetization Timeout	3 x character times	 
AP API Enable	API enabled [1]	 

Figure 4: Serial Configuration Settings of Xbee

3 Xbee Communication

To start this analysis, I will first look at the method of wireless communication employed by the system. Xbees are a commonly used wireless device that uses a protocol called Zigbee. As each major system of the project makes use of these devices to interface with the others, those interfaces will be clearer with an understanding of the Xbees. This section assumes the setup from section 2.5 has been handled correctly, with the appropriate pairings between device pair.

Communication with an Xbee can be handled via Serial commands, which Python is capable of sending. The process of establishing communication between the Raspberry Pi running a Python script and the Shuttle involved a number of research dead ends before arriving at a fairly simple solution. There are a few libraries meant to facilitate the control and configuration of Xbees, particularly in a network or the less intuitive Xbee Series 2. Since the system uses 2 pairs of Xbee 1's, the communication is direct, and there is no need for any sort of configuration commands to be passed to the Xbee. I need only send a string of characters for it to send to its partner. By turning the Pi's Xbee to transparent mode, any serial data it receives (which is sent via PySerial functions) is directly sent to the partner. The link between Pi and Xbee is made using an Xbee explorer board, which connects via USB.

In order to confirm the behavior of the Xbee, I had to make it behave as the controller's Xbee, sending only a single character as a directional command. In order to understand the controller, prior to receiving the code which it is running, I used X-CTU to monitor the data received by the corresponding Xbee on the shuttle. Table 2 was created prior to the code being received, and used to start spoofing the controller with Python. By changing the destination of the Pi Xbee to match the controller's, it was possible to send the "L" character on a loop to activate the corresponding fan. The code for this communication is shown below in code snippet 1.

```
## Serial Library Allows communication over pins or USB devices
import serial
##Sets the serial port as the USB port which Xbee is plugged into
PORT = '/dev/ttyUSB0'
## Baud rate which Xbee is configured to read/send data at
BAUDRATE = 115200
# Open serial port
ser = serial.Serial(PORT, BAUDRATE)

# Continuously print packets to serial port
while True:
    try:
        ser.write('L') #immitate sending left command from the controller
        ## Example of autopilot command
        ## ser.write('A0B0C255D0E255F0Z')
    except KeyboardInterrupt:
        ser.close()
        break
```

Listing 1: Python Xbee Communication Example

4 Joystick Controller

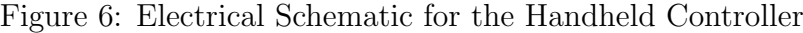
An initial step to understanding the existing system was to establish electrical diagrams and detailed code of the controller (Fig. 5). This was particularly the case prior to receiving the code which is run on the Arduinos, as it allows for understanding of the shuttle by extension. With an understanding of what the controller is doing, I was able to simulate it's behavior to test the Raspberry Pi communication, mentioned in more detail later.



Figure 5: Handheld Controller

4.1 Electrical Schematic

The electrical schematic allows us to understand the high level interactions of the controller, along with the specific use of each pin within the Arduino Uno, which make's the code clearer. Of particular note in the evaluation of the electronics is that the controller uses joysticks that behave as switches. Each of the 2 joysticks has 4 limit switches which are activated (closed) when the stick is moved in the corresponding direction. The inputs from said joysticks are converted to commands which are sent by the Xbee wirelessly to the shuttle. The Xbee's Tx and Rx pins are connected to the Rx and Tx pins of the Uno, which allows for serial communication between the two. The schematic for the controller is shown below in Fig. 6. Each of the switches for the joysticks is connected via pull-down resistors to ground. Their values are different likely since they happened to be on hand when constructing the circuit, since the resistance is not critical to their behavior of keeping the voltage of the pins at GND except when a switch is activated.



4.2 Commands and Control

The evaluation of commands sent by the joystick controller was the next step in grasping what it and the system were doing. Prior to having the code to look at, I sought to see what the controller's Xbee was sending the shuttle's. That endeavor (discussed in more detail in the above section on Xbee communication) resulted in the following table, which was later corroborated by investigating the code. Table 2 shows what each input results in for data sent via Xbee.

Table 2: Input and Corresponding System Command

Input	Xbee Command	Meaning
Toggle Up (Pin 8)	X	Auto-Pilot
2	L	Move Left
3	D	Move Back
4	R	Move Right
5	U	Move Forward
6	Q	Rotate CCW
7	E	Rotate CW

This means that the manual control has digital input in each direction, which is fine given the speed of the system, but the auto-pilot requires a different mode of communication and control, since I want a smoother analog input. The aforementioned code is relatively simple, using just a few "if" statements to translate from a digital pin being active to sending a character. code snippet 2 demonstrates how the controller Arduino handles inputs and sends messages via the connected Xbee to the system. If the toggle switch is set to manual, then each other digital pin is read to determine if any toggle switches have been activated by the joysticks. If any of those switches is active, and the digital pin is read as high, then the corresponding character is sent via the serial pins to the Xbee. If the toggle is in Auto-pilot mode, then "X" is sent, signaling the shuttle to look for instructions from the other Xbee.

```

1 void loop() {
2   if (digitalRead(8) == HIGH) { //If the Toggle is in the Down "Manual
      " position
3     //Series of if statements to determine which direction joystick is
      aimed
4     if (digitalRead(2) == HIGH) {
5       // Serial.println("Move Left");
6       Serial.println("L"); }
7     ...}
8   else {
9     //If the toggle is up "Autopilot" then send X, indicating...
10    that instructions should be taken from the other Xbee
11    Serial.println("X"); } }

```

Listing 2: Arduino Code For Joystick Controller

5 Shuttle System

Next, I look at the space shuttle system itself (Fig. 7). As a general overview, this system is made up of an MCU (Arduino Mega) which takes in wireless communication from the Xbees and uses that data to activate fans which push it over an air table. This system is quite a bit more complicated than our joystick controller in both wiring and software, since the system must control six fans and receive commands from two Xbees.

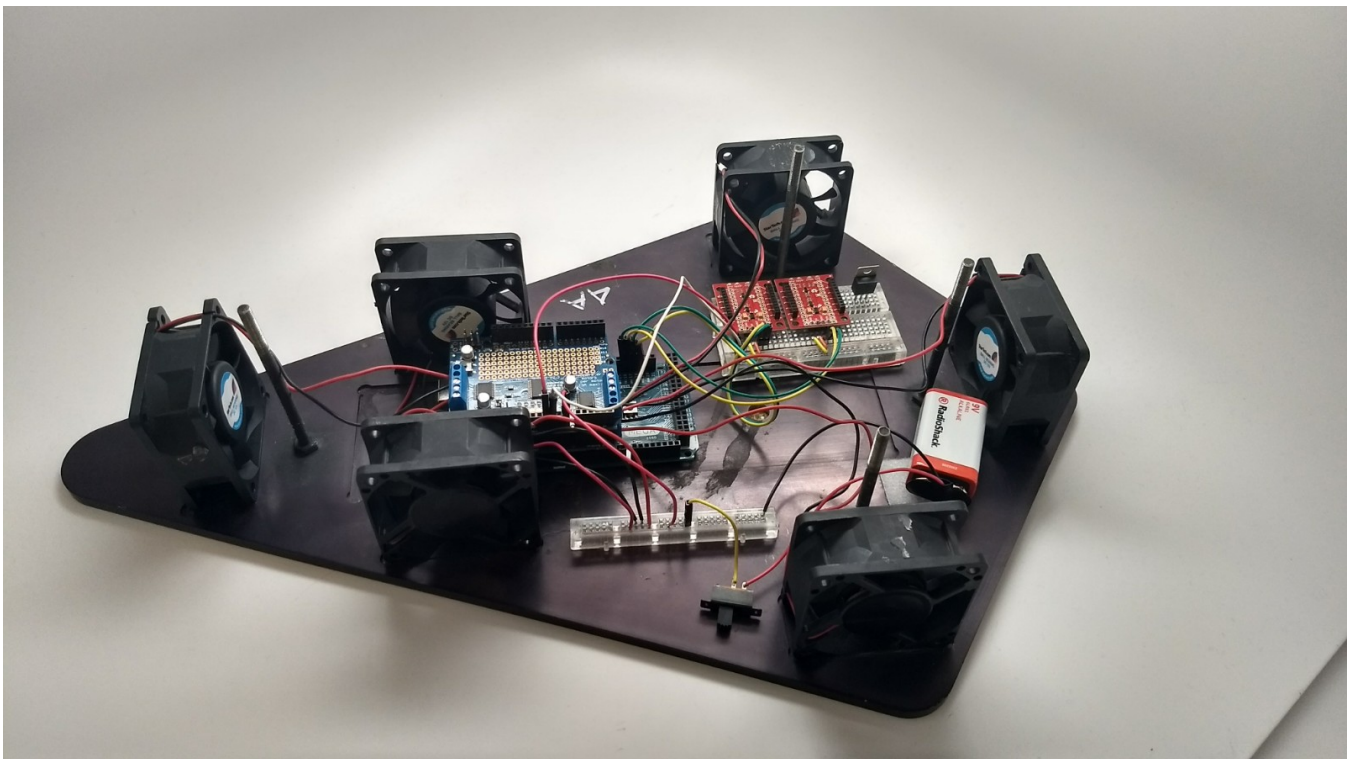


Figure 7: Model of the Space Shuttle

5.1 Electrical Schematic

The use of Arduino motor shields here makes for an fairly simple implementation of the DC case fans used for propulsion. The schematic for the system is shown below in Fig. 8. The shields simply plug in atop the Mega board, which allows them to communicate with minimal effort. An important note is that the battery is powering both the Arduino Mega, and directly powering the motors through the motor controllers. This means that the voltage of the power that the motors receive is not dropped to 5v as with peripherals connected to the Mega's 5v out pin, such as the Xbees. The power of the system is provided by a 9V battery, which I propose changing in the following section. The Xbees are connected in the same manner as the one on the joystick controller, though the Mega has two sets of Rx and Tx pins, which conveniently allows us to use the two channels.

5.2 Battery Improvements

Recommended operating voltages for the Mega board are between 6v and 12v, so I wanted to switch to a different battery with higher capacity than the average 9v. A higher voltage is also desirable since the fans, which are directly powered by the battery are meant to receive 12v, and will operate at higher speeds. A normal capacity for a 9v, as used on the system currently, is about 600 mAh. The Mega's datasheet states that each I/O pin, of which 4 are active at all times, consumes 40 mA, and the fans have a max current draw of 90 mA each. Given the nature of the code for control discussed above, only 1 motor is ever active at a time, however I plan on eventually being able to use 3 fans at once to position the system. The continual use battery life of our system is calculated in Eqn 1

$$\begin{aligned} I &= N_{\text{fan}} * I_{\text{fan}} + N_{\text{pin}} * I_{\text{pin}} = 3 * 90\text{mA} + 4 * 40\text{mA} = 430 \text{ mA} \\ \text{Life} &= \frac{C}{I} = \frac{600\text{mAh}}{430\text{mA}} = 1.4\text{hours} \end{aligned} \tag{1}$$

This must obviously be improved upon, as the exhibit will need to active for more than 1.4 hours in a given day. The recommended input voltage for the Arduino Mega is between 7-12V, which gives us a few options for batteries. One of the most appealing options is a Li-Po battery pack, which come in a variety of configurations that meet our specs without being too expensive or heavy. One example is a 5000 mAh pack from Turnigy, which operates at 11.4V and weighs 0.78 lbs. This is about 8 times heavier than the 9V, but is, given the overall weight of the system, a worthy trade-off for an eight-fold increase in battery life, which would get us through the day even constantly moving the system. This battery has been added to the system, and greatly improved the output force of the fans. The initial fan force calculations described in the next subsection were done with the 9V battery, and the improvement will determined through further system ID after the testing described at the end of this report.

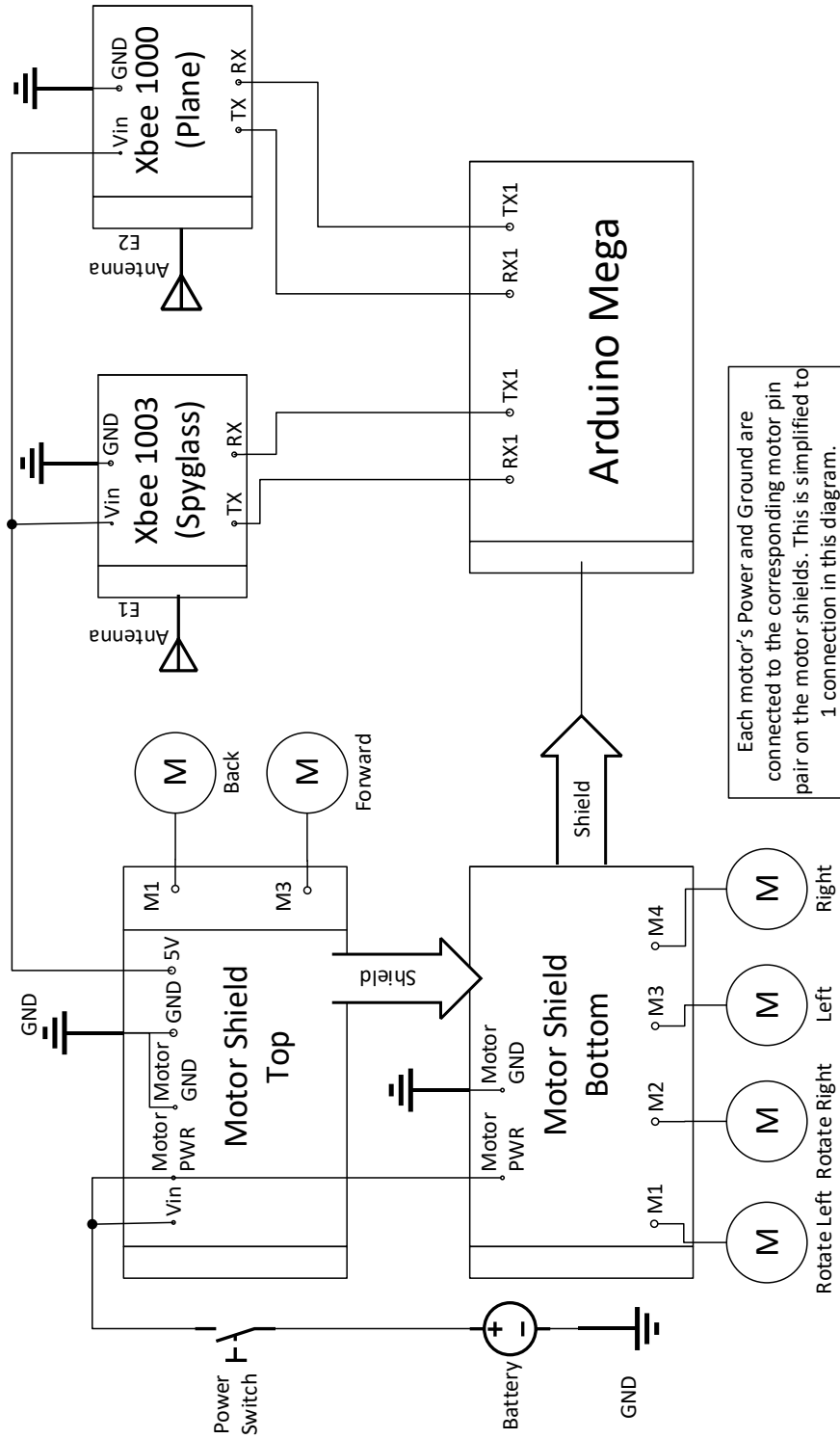


Figure 8: Electrical Schematic for the Shuttle model

5.3 Physical System ID

5.3.1 Thrusters (Fans)

The first step taken in understanding the physical system of the shuttle was determining how the thrusters, or fans, responded to inputs from the arduino controller. How the motor speeds are set in control is described a bit later, but for now what is important is that the motors take a speed setting of 0-255 from the controller and motor shield, and produce different levels of air flow, and therefore thrust. Using small desk case fans, specifically StarTech 80mm case fans, means the system is receiving minute forces even at full power, which makes for a difficult time measuring the forces and relationship between force and input value. Initially I attempted to use a large load cell and a frictionless cart, but the fan could not provide enough force to overcome the "frictionless" resistance of the cart. This led me to the using a much simpler setup and smaller load cell with a capacity of 2 lbs, providing relatively good resolution.



Figure 9: Calibration Testing of the Load Cell

In order to calibrate the load cell, I used the sets of masses from the ITLL to establish the relationship between weight and voltage measured by the load cell. One of those experiments is shown in Fig. 9. With that relationship in hand, the next step was to hook the fan up to the load cell and measure it's output force at various motor speed settings. The fan was simply secured to the end of the load cell, and behaved as the weights did in the prior experiment. This setup is shown in 10. The resulting data was collected and analyzed to determine what the function that relates the speed and thrust is. The line of best fit turned out to be a second order polynomial, as shown in Fig. 11, however a linear model, as shown in Fig. 12, was selected. This was due to later complications with simulation, and the improved speed of calculation. This model may prove to be slightly less accurate, though do to the saturation discussed later, this is unlikely to be a significant problem. This function is the "actuator" portion of the standard feedback loop, and will allow me to calculate the speed required to be sent to each fan given the desired forces on the system.

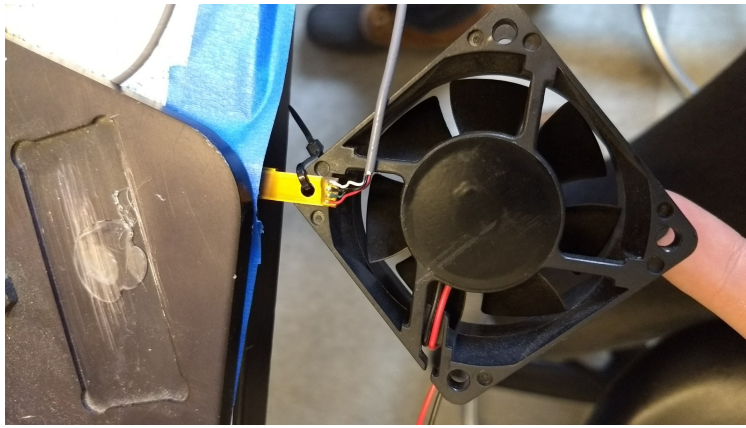


Figure 10: Load Cell Testing of the Fan

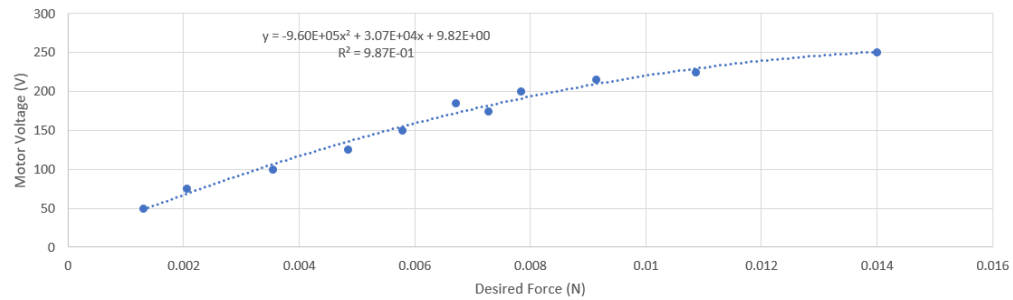


Figure 11: 2nd Order Relationship Between Desired Force and Motor Speed

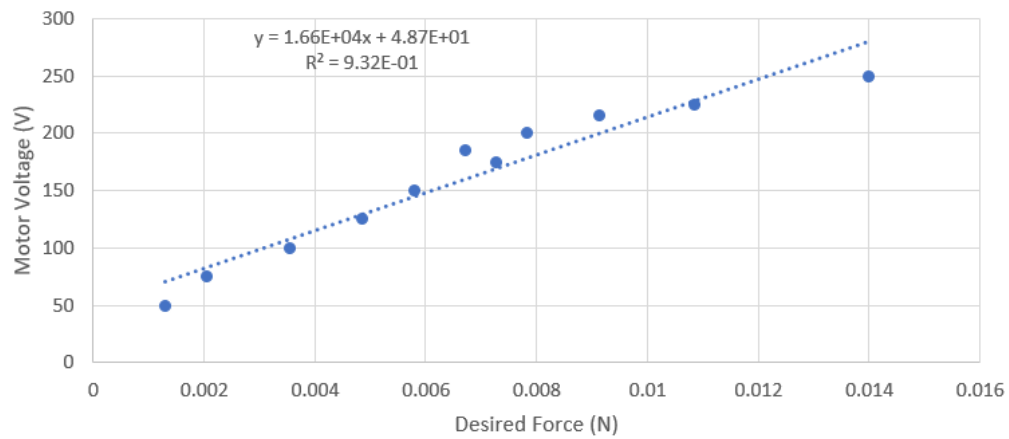


Figure 12: Linear Relationship Between Desired Force and Motor Speed

5.3.2 Physical Model

A CAD model of the system was created to evaluate the physical properties of the system such as center of mass and moment of inertia for rotation. This CAD model is shown below in Fig 13. This model places the battery forward, against the Mega, which moves the center of mass between the two side fans, which is a departure from the previously expected position. This CoM is only an estimate due to modeling errors, but the center of mass being moved forward by the battery will nonetheless allow a significant improvement to the commands to move sideways and rotate, since now two fans may be used to rotate or strafe sideways, effectively doubling the response in those situations. This change, along with modeling the system allowed for an estimation of the governing physical equations of the system. The equations that were derived from this model are shown in the controls section where I discuss simulation and it's implications further.

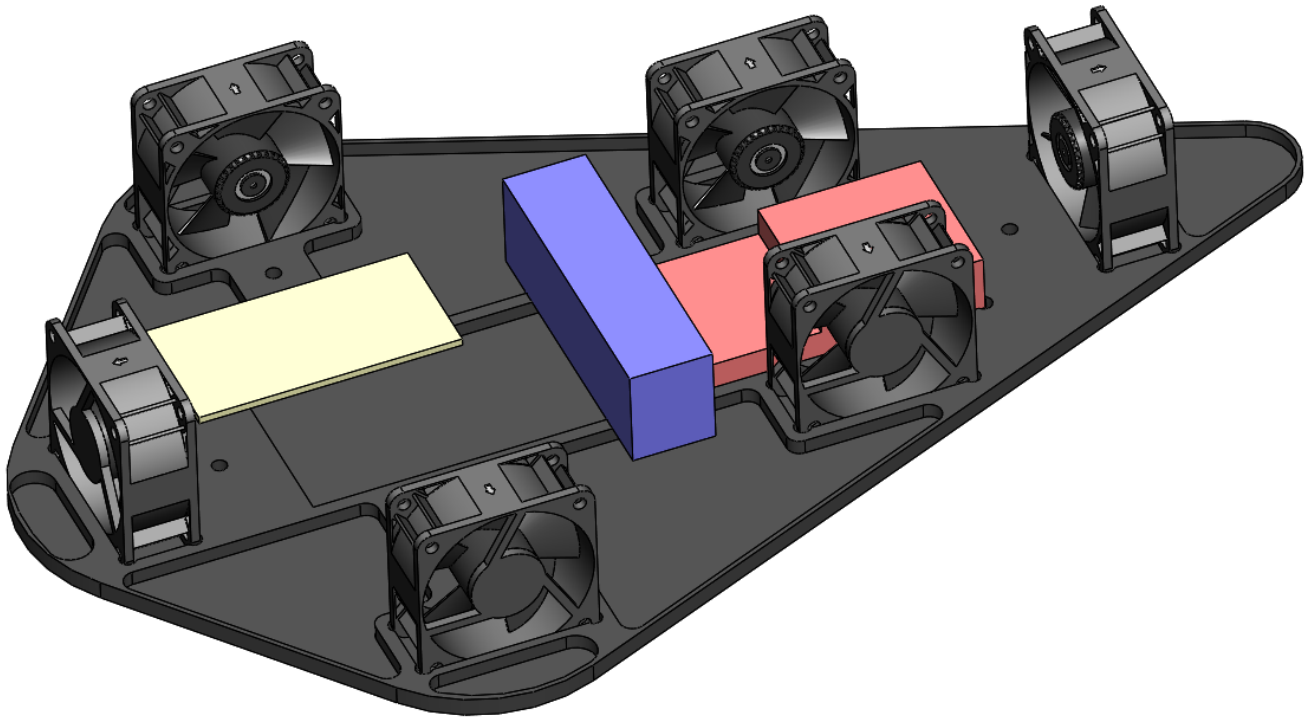


Figure 13: CAD Render of Shuttle

5.4 Receiving & Interpreting Wireless Commands

There are two major functions performed by the Arduino aboard the shuttle model. First is to receive the commands from the controller and execute controls according to the characters received. Second is to conduct the auto-piloting given data about the position of the shuttle. I will discuss the second point in the control section in more detail. code snippet 3 shows how the motors are initiated, and that each of the variables of names like "TurnLeft" are used to define the output speed. Previously, the rear side fans were used for lateral movement and the forward side fans used for rotation. After modeling the system, it became apparent that the center of mass was centered such that both side fans should be used for lateral movement and opposing fans used for rotation. This change is reflected in the code shown throughout.

```
1 // Establishes the motor control variables using the 2 motor shields
2 Adafruit_MotorShield AFMSbottom = Adafruit_MotorShield(0x60);
3 Adafruit_DCMotor *RightFront = AFMSbottom.getMotor(1);
4 Adafruit_DCMotor *LeftFront = AFMSbottom.getMotor(2);
5 Adafruit_DCMotor *LeftBack = AFMSbottom.getMotor(3);
6 Adafruit_DCMotor *RightBack = AFMSbottom.getMotor(4);
7 Adafruit_MotorShield AFMStop = Adafruit_MotorShield(0x61);
8 Adafruit_DCMotor *Bwd = AFMStop.getMotor(1);
9 Adafruit_DCMotor *Fwd = AFMStop.getMotor(3);
```

Listing 3: Motor Shield Initialization

With this setup, activating and controlling the fans is a simple matter, as shown in the next code snippet. This is the first of the If statements that correspond to the possible commands from the controller. For each, if the character read from the Xbee's serial connection matches a command, then the corresponding fan speeds are set to high and activated.

```
1 if (Serial2.available()) {
2     //If there is a message from the controller Xbee
3     dir = Serial2.read();
4     Serial.println(dir);
5     if (dir == 'R') { //If the controller sent the signal 'R'
6         RightBack->setSpeed(0);
7         Fwd->setSpeed(0);
8         Bwd->setSpeed(0);
9         RightFront->setSpeed(0);
10        //Set motor to full power
11        LeftFront->setSpeed(255);
12        //Run the motor forward at the set speed
13        LeftFront->run(FORWARD);
14        LeftBack->setSpeed(255);
15        LeftBack->run(FORWARD);
16        Serial.println("Controller_Right"); }
```

Listing 4: Motor Activation

If the "X" command is received, the system then proceeds to look for data from the other Xbee, which receives auto-piloting commands of motor speeds. This command string makes use of certain characters as flags to mark the beginning and ending of a command. This is a great way to ensure that even if multiple commands are waiting in the buffer, the code is executed correctly and orderly.

```
1 if (dir == 'X') {
2     //If the toggle is set to auto-pilot control
3     Serial.println("Computer_Time");
4     // Print to terminal that the auto-pilot is taking over
5     if (Serial1.available() > 0) {
6         // If there is data from the Pi Xbee
7         val = Serial1.read();
8         Serial.print(val);
9         if (val == 'A') { //Initiator symbol for a new command
10             for (int i = 0; i < 100; ++i) {
11                 //Read in the characters of the string
12                 val = Serial1.read();
13                 if (val == 'Z') {
14                     //If the end symbol '>' is read, end the loop
15                     ReadData = String(ReadData + val);
16                     Serial.print("Read_Data:");
17                     Serial.println(ReadData);
18                     // Print the full string to the console
19                     break;
20                 }
21                 else {
22                     ReadData = String(ReadData + val); }
23                 // Concatenate the new char to end of the string
24             }
25             ReadDataOld = ReadData; //Save the newly read data
26         }
27     }
28     else {
29         ReadData = ReadDataOld;
30         //If no new data, then use the same data
```

Listing 5: Command String Saving

With the command read in, it is then further broken down and translated into the individual variables that correspond to the motor speeds. This is done by converting a segment of the string, up until the following flag, into an int which can be passed to the motor control function. This decomposition is shown in code snippet 6. After this point, the motor speeds are sent to the corresponding motors until the next loop.

```
1 //With Data read in, set the delta values from the string
2 for (int i = 0; i < 50; ++i) { //Go through the data string until
3     if (ReadData[i] == 'A' ) { //If the start character is read
4         Control_string = "";
5         ++i; //Skip the A character
6     }
7     else if (ReadData[i] == 'B') {
8         // If b character is read, convert current string to v[0]
9         v[0] = Control_string.toInt();
10        Control_string = "";
11        ++i; //Skip the X character to start reading the first v value
12    }
13
14    ...
15
16    else if (ReadData[i] == 'Z') {
17        // If z character is read, convert current string and exit loop
18        v[5] = Control_string.toInt();
19        Control_string = "";
20        break; //Exit this loop as data has been extracted from the
                string
21    }
22    // Add the characters to the string
23    Control_string = String(Control_string + ReadData[i]);}
```

Listing 6: Command String Interpretation

6 Computer Vision

As has been mentioned throughout this report, Computer Vision (CV) will be used for the detection or 'sensing' of the system to inform the controller. Computer vision is the use of software to detect objects or patterns within an image, which will be a live feed from an attached camera in this case. My involvement in this project stems from taking the computer vision course, in which I chose to begin this project as my final for the class. Since then, a number of improvements have been made. The flow of the python script is shown in Fig. 14. In the proceeding section I will describe the detection algorithm, among other considerations for the python code.

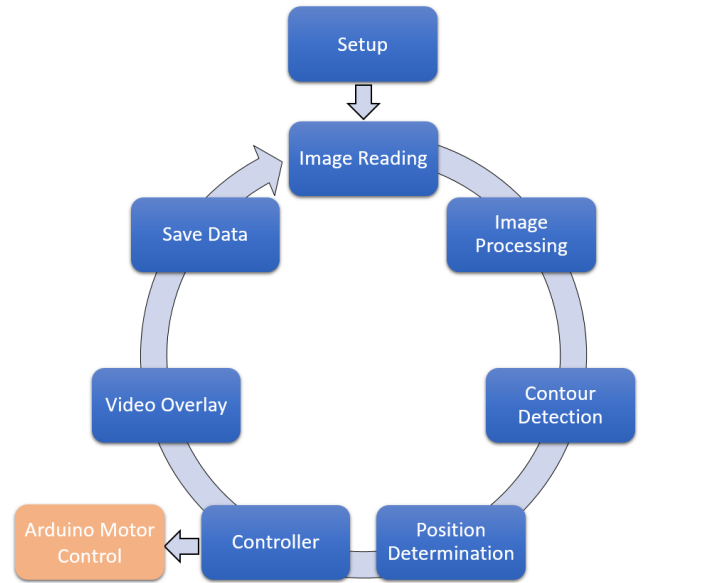


Figure 14: Flow of Python CV Code

6.1 Camera and Image Settings

The critical physical component of computer vision is the camera which takes images for processing. In this project, I am using a PiCamera, a relatively cheap but well supported camera meant to interface with the Raspberry Pi on which the computer vision code is being run. This camera does not have a zoom feature, which lead to a question of whether I might need a different camera due to the distance between the ceiling of the exhibit and the object, which is around 10 ft. The full field of view (FOV) of an image taken from 12 ft away is shown in Fig. 15, where you can see that much beyond the desired object would be visible in frames.

To handle the large FOV which this distance carries, I take an image and then focus on the center of the image for processing so that the fringes which do not contain the object are eliminated. The resulting focused image is still the same resolution in terms of pixels per distance, but is much smaller and therefore quicker to process. Code snippet 7 shows how the image is captured, and then reduced in size to the center half of the original. The reduced size will be apparent in proceeding figures. One other important point in this snippet is the use of the video port of the PiCamera as opposed to the image port. This results in slightly lower quality images, but a significant uptick in speed of capture from around 20 to 80 Hz.



Figure 15: Original Image Taken 7 ft Above Object

```
def setup():
    # Defining the size of the image the camera will take and camera settings
    global rows, cols, imagefocus, portion, offset
    rows = 120
    cols = 160
    portion = 4 # Define the range of the image which we care about
    # Define the portion as 2/'portion' of img (i.e. center half is portion=4)
    imagefocus = np.array(
        [[cols / 2 - cols / portion, cols / 2 + cols / portion],
         [rows / 2 - rows / portion, rows / 2 + rows / portion]])
    # Camera Settings
    global cap
    cap = cv2.VideoCapture(0)
    ret = cap.set(cv2.CAP_PROP_FRAME_WIDTH, cols)
    ret = cap.set(cv2.CAP_PROP_FRAME_HEIGHT, rows)
    ...
def Image_Read():
    global img, image_focused, image
    ret, frame = cap.read()
    image = frame.reshape((rows, cols, 3))
    image_undistorted = cv2.undistort(image, mtx, dist, None)
    image_focused = image_undistorted[imagefocus[1, 0]:imagefocus[1, 1],
                                      imagefocus[0, 0]:imagefocus[0, 1]]
    img = cv2.cvtColor(image_focused, cv2.COLOR_BGR2GRAY)
    return
```

Listing 7: Camera Settings

6.2 Object Detection

With the focused grayscale image captured, I can begin the true computer vision aspect of this project: detecting the position and orientation of the shuttle. There are 2 major steps that occur, but use very similar methodology. Firstly, upon initialization of the python program, the target destination and orientation are found. After that, the main loop is entered and a slightly different process finds the position and orientation of the shuttle. Due to the similarity, only the code for shuttle detection will be discussed in detail, but where the differences in the two detection methods arise will be pointed out.

6.2.1 Thresholding and Object Refinement

The first step in detection is to perform a thresholding operation to separate the object from the background. The built in cv2 threshold method is used. This operation then sets the value for every pixel whose brightness value is above a threshold to white, and all those below to black, creating a binary image highlighting the objects of interest. This is a very simple and effective process for our case because the object and target are both stark white against a black background. This results in both the shuttle and the target parking space outline being shown against a black background, though the shuttle does contain holes from black trim. In order to make these shapes singular blobs, a 'closing' operation is used on the objects. Closing can be thought of as filling in holes, because it first adds a layer of pixels around the edge of each shape (dilation) which can fill in small gaps, and then eliminates the extra layers of pixels along the outside of the filled shape (erosion). When a closing operation is performed on the threshold image, the image Fig. 16 is created. The code for this process, used for initial target location, is shown in code snippet 8.

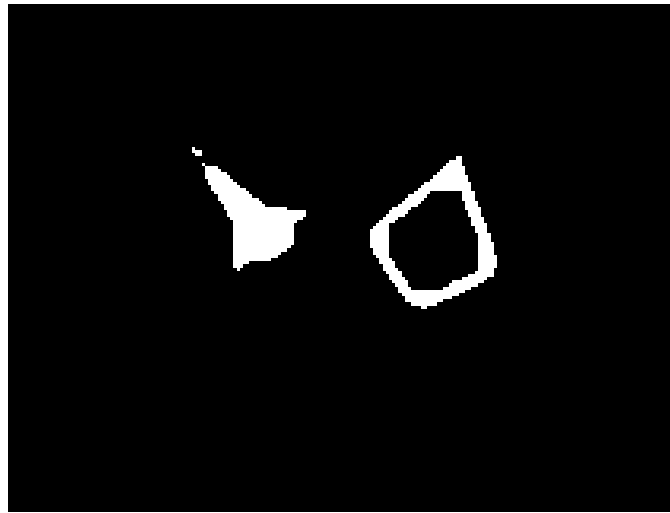


Figure 16: Image After Otsu Threshold and Closing

```
TH, imgTH = cv2.threshold(img, 70, 255, cv2.THRESH_BINARY)
cv2.imshow("THed", imgTH)
# Close the image to remove the shape of the parking space
closed = cv2.morphologyEx(imgTH, cv2.MORPH_CLOSE, kernel5, iterations=1)
```

Listing 8: Thresholding and Closing

The thresholded image is used to find the orientation and position of the parking space, visible on the right of Fig 16. When detecting the position of the shuttle however, the target is unnecessary and potentially problematic. Therefore, I do not perform a closing during the main loop, and instead erode the image operation to eliminate the target from the picture. Code snippet 9 shows the erosion operation which removes smaller objects. When eroding (which removes a layer of pixels), I perform the process once with a larger kernel to eliminate all trace of the parking space, but leave enough of shuttle. The result of thresholding and then eroding the focused image is shown in Fig. 17.

```
## Erode the image to remove the shape of the parking space
eroded = cv2.morphologyEx(imgTH, cv2.MORPH_ERODE, kernel7, iterations=1)
```

Listing 9: Image Erosion

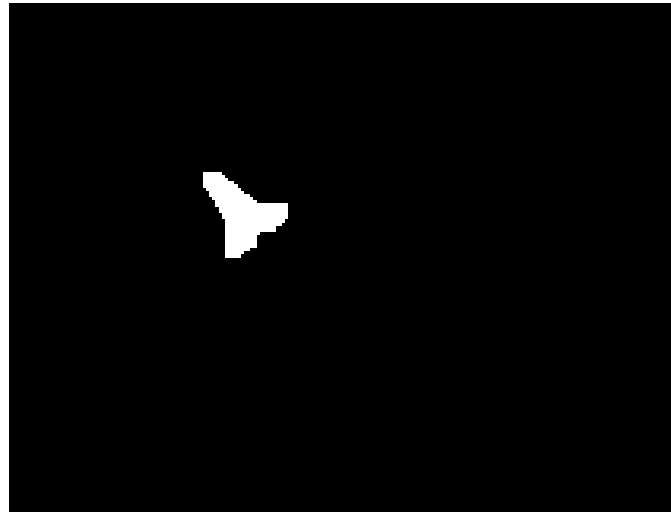


Figure 17: Shuttle After Otsu Threshold and Erosion

6.2.2 Position and Orientation Detection

Next, I find the centroid of the object(shuttle or parking space) which will be treated as it's coordinates. The openCV contour functions are used in the initialization for robustness of detecting the target, but a customized edge detection is used in the main loop for the shuttle. The result of using findContours is a set of contours for each significant object in the thresholded image used as an input. The contours of both the shuttle and target are shown in Fig. 18.

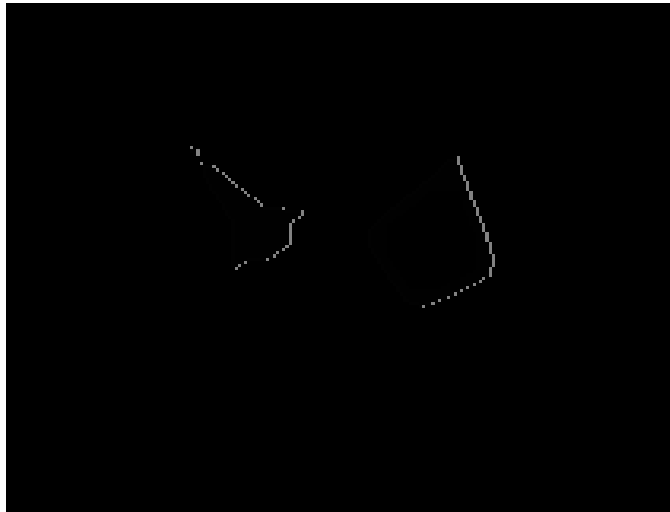


Figure 18: Shuttle and Target Contours

These contours are essentially the edges that define the shapes, and can be used in conjunction with other commands to quickly ascertain important data. The moments function takes a contour and returns quite a bit of information regarding the moment of inertia of the object, which can be used to determine the centroid. The code used to detect the shuttle is similar, but relies less on nebulous functions to perform essentially the same task. Additionally, given the nature of the kite shapes I am detecting, a very simple method of determining orientation can be used. I take the centroid of the object's contour, then determine which point along the contour is furthest (highest norm) from the centroid and define that as the tip of the shape. With a tip and center defined, a line can be drawn between the two, and the inverse tangent used to determine the orientation of the line. The code used for Target detection is shown in code snippet 10, and Shuttle detection with Sobel edge detection is shown in code snippet 11. Fig. 19 shows the calculated centroid and axial line overlaid on the original image of the ship and target. With the relative position and attitude detected, the next step is to use those values to control the shuttle.



Figure 19: Tracked Shuttle and Target Overlaid with Position and Attitude

```

def Target_Detection():
    global Target, tip_Target, theta_Target, Rotation
    # ——— Use detection once to find the parking spot ———
    ret, frame_o = cap.read()
    image = frame_o.reshape((rows, cols, 3))
    # Converts img data to grayscale, Cuts image down to focus area
    imageGray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    img = imageGray[imagefocus[1, 0]:imagefocus[1, 1],
                        imagefocus[0, 0]:imagefocus[0, 1]]

    # Threshold the image
    TH, imgTH = cv2.threshold(img, 100, 255, cv2.THRESH_BINARY)
    # Closing the image to complete the shape of the shuttle
    closed = cv2.morphologyEx(imgTH, cv2.MORPH_CLOSE, kernel5, iterations=1)
    #cv2.imshow(" Objects", closed)
    # Find the Contours of the img, then specifically calling out the longest
    one
    im_dummys, contours, hierarchy = \
        cv2.findContours(closed, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    cnt = contours[np.argmax(len(contours[:]))]
    # Moment and centroid definition
    M = cv2.moments(cnt)
    Target = [int(M['m10'] / M['m00']), int(M['m01'] / M['m00'])] # y, x
    # Finds the avg of the contour, defines tip as furthest from the avg.
    # This calc is only useful because of shape of ship
    dif = np.subtract(cnt[:, 0], Target[:, 0])
    normal = np.sqrt(np.add(np.multiply(dif[:, 0], dif[:, 0]),
                                       np.multiply(dif[:, 1], dif[:, 1])))
    tip_Target = cnt[np.argmax(normal)][0]
    # Angle calculation based on slopes of line
    dy = -Target[0] + tip_Target[0]
    dx = -Target[1] + tip_Target[1]
    theta_Target = math.degrees(math.atan2(dy, dx))
    Rotation = np.array(
        [[math.cos(theta_Target), -math.sin(theta_Target)],
         [math.sin(theta_Target), math.cos(theta_Target)]]
    )
    return

```

Listing 10: FindContour() Position detection

```

def ContourDetection_Sobel():
    global center, tip
    ##### Find the Edges of the img, then put all of the edge pts into a vector
    try:
        sobelx = cv2.Sobel(eroded, cv2.CV_64F, 1, 0, ksize=3)
        sobely = cv2.Sobel(eroded, cv2.CV_64F, 0, 1, ksize=3)
        sobelMAG = np.uint8(np.sqrt(np.add(np.multiply(sobelx, sobelx),
                                                    np.multiply(sobely, sobely))))
        cv2.imshow("Sobel_Edge_Detector_Mag", sobelMAG)
        silhouette = np.array(np.where(sobelMAG > 100))
        # Calculate centroid (avg pt from edge) in y,x
        silhouette_length = silhouette.shape[1]
        sum_x = np.sum(silhouette[1, :])
        sum_y = np.sum(silhouette[0, :])
        center = [sum_x/silhouette_length, sum_y/silhouette_length]
        # Defines the tip coords as the furthest from the center.
        # This calc is only really possible because of shape of ship
        dif = np.array([np.subtract(silhouette[0, :], center[1]),
                        np.subtract(silhouette[1, :], center[0])])
        normal = np.add(np.multiply(dif[0, :], dif[0, :]),
                        np.multiply(dif[1, :], dif[1, :]))
        tip = silhouette[:, np.argmax(normal)]
    except:
        print("Sobel_Line_Detection_Failed")
        ContourDetection_findContours() #Try this instead
    return

```

Listing 11: Sobel Edge Position Detection

6.3 Alternative Tracking Method

An alternative method for tracking the object I wanted to test was to use colors as object points rather than tracking the actual shape of the shuttle. I used colors which were far enough apart on the HSV hue scale to be distinguishable, choosing magenta for the center, yellow for the tip, and blue for the wingtips. The layout and colors are shown in 20. The code for tracking via specific colors is shown in code snippet 12. This method uses HSV images rather than grayscale, and creates a separate mask for each color, meaning there is 3 times more data process. With the speed of the overall tracker, this may be acceptable, but even if not, it presents a great alternative for augmented reality projections, as discussed in Section 6.4.2.



Figure 20: Colors for Alternative Tracking

```
def setup():
    # Declare HSV marker values
    global Cyan_MIN, Cyan_MAX, Magenta_MIN, Magenta_MAX, Yellow_MIN, Yellow_MAX
    # Cyan - tip
    Cyan_MIN = np.array([80, 70, 50], np.uint8)
    Cyan_MAX = np.array([130, 255, 255], np.uint8)
    # Yellow - center
    Yellow_MIN = np.array([20, 50, 50], np.uint8)
    Yellow_MAX = np.array([60, 255, 255], np.uint8)
    # Magenta - wingtip
    Magenta_MIN = np.array([140, 30, 40], np.uint8)
    Magenta_MAX = np.array([180, 255, 255], np.uint8)
    ...
def ContourDetection_findContours():
    global center, tip, wing
    im, Cyan_contours, h = cv2.findContours(Cyan_threshed,
                                             cv2.RETR_LIST, cv2.CHAIN_APPROX_NONE)
    im, Yellow_contours, h = cv2.findContours(Yellow_threshed,
                                              cv2.RETR_LIST, cv2.CHAIN_APPROX_NONE)
    im, Magenta_contours, h = cv2.findContours(Magenta_threshed,
                                              cv2.RETR_LIST, cv2.CHAIN_APPROX_NONE)

    try:
        Yellow_cnt = sorted(Yellow_contours, key=cv2.contourArea, reverse=True)[0]
        M = cv2.moments(Yellow_cnt)
        tip = [int(M['m10'] / M['m00']), int(M['m01'] / M['m00'])] # y, x
        Magenta_cnt = sorted(Magenta_contours, key=cv2.contourArea, reverse=True)[0]
        M = cv2.moments(Magenta_cnt)
        center = [int(M['m10'] / M['m00']), int(M['m01'] / M['m00'])] # y, x
        Cyan_cnt = sorted(Cyan_contours, key=cv2.contourArea, reverse=True)[0]
        M = cv2.moments(Cyan_cnt)
        wing = [int(M['m10'] / M['m00']), int(M['m01'] / M['m00'])] # y, x
    except:
        print "Something_broke_the_contours"
    return
```

Listing 12: Alternative Position Detection: Color Code

6.4 Augmented Reality Overlay

The final use of CV in this project does not pertain to controlling or speeding up the system, but rather showing a outside observer what the controller is doing. There were 3 things which I added to the real image, aiming to harness the power of computer vision as an informative tool.

6.4.1 Booster Flames

The motor control values, the determination of which is discussed in section 7, correlate to the strength of the "boosters" on the shuttle. In order to show the active efforts of the boosters in controlling the shuttle, I project scaling flame images on top of active fans. In order to accomplish this projection, I import an image of a flame and set the opacity of background pixels to 0, and foreground (colored) pixels to 255. A standard mapping of the shuttle was created so that the location of the anchor points, where the thrusters are nominally located, can scale and be placed appropriately on the shuttle in whatever orientation and size we see it. The setup code for flame projection is shown in code snippet 13.

```
def setup():
    # Import Flame and set alphas, then create rotated versions
    global jet_scale, Flame_front, Flame_rear, Flame_left, Flame_right
    jet_scale = 10
    Flame_imported = cv2.imread("Flame.png",-1)
    Flame = cv2.resize(Flame_imported, dsize = (cols,rows))
    # Set Opacity of Colored Cells to max
    non_white_cells = np.where(Flame != [255,255,255,0])
    Flame [non_white_cells [0]][non_white_cells [1]][3] = 255

...

#location of jet anchors in x,y at standard length (26.45 pixels)
global standard_length, standard_wingspan, Anchors
standard_length = 26.45
standard_wingspan = (standard_length / real_length) * real_wingspan
Right_rear = [-15,-5]
Right_front = [-5,15]
Left_rear = [-Right_rear [0],Right_rear [1]]
Left_front = [-Right_front [0],Right_front [1]]
Front = [0,25]
Rear = [0,-25]
Anchors = np.array ([Right_front ,Left_front ,Right_rear ,Left_rear ,Rear ,Front])
```

Listing 13: Flame Import and Anchor Point Setup

For projection, the image of the shuttle is first rotated so that it is facing downwards. This allows for fewer rotation actions and less math than rotating the flames themselves since there could be up to 4 active flames but this will approach always rotate the whole image twice. Once the shuttle is in a standard position, I use the length from center to tip compared to the "standard" length where I created the anchor points to scale up or down where the flames should be placed. next, every active booster has a flame projected on the corresponding anchor point. This is where the opacity comes into play, only adding the pixels from the flame, but not the background of that image. Finally, the modified image is rotated in the opposite direction, resulting in a new version of the real image with the projected flames. The code for this projection is shown in code snippet 14 and the resulting image in Fig. 21.

```

def Flame_Projection(): # Flame Projection
# Create array of anchor points as they relate to a downward facing shuttle
anchor_points = np.divide(Anchors,(standard_length/ship_length))
# Rotate image to figure out layout
Ship_Rotation = cv2.getRotationMatrix2D((center[0],center[1]),-theta,1)
rotated = cv2.warpAffine(image_focused,Ship_Rotation,
                        (int(cols*2/portion),int(rows*2/portion)))
try: # Projected Flames
if v[5] > 0: # Front
    Scaled_Flame = cv2.resize(Flame_front, dsize = (0,0),
                              fx = v[5]/(jet_scale*255.0), fy = v[5]/(jet_scale*255.0))
    Flame_size = np.shape(Scaled_Flame)
    alpha_s = Scaled_Flame[:, :, 3] / 255.0
    alpha_l = 1.0 - alpha_s
    TL = np.array([anchor_points[5][0]+center[0]-Flame_size[1]/2+offset,
                  anchor_points[5][1]+center[1]]).astype('int16')
    BR = np.add(TL, [Flame_size[1] + offset, Flame_size[0]]).astype('int16')
    for c in range(0, 3):
        rotated[TL[1]:BR[1], TL[0]:BR[0], c] =
            (alpha_s * Scaled_Flame[:, :, c] +
             alpha_l * rotated[TL[1]:BR[1], TL[0]:BR[0], c])
...    # Repeat for each flame
Ship_Rotation = cv2.getRotationMatrix2D((center[0],center[1]),theta+1.25,1)
unrotated = cv2.warpAffine(rotated,Ship_Rotation,
                        (int(cols*2/portion),int(rows*2/portion)))
cv2.imshow('projected_and_returned_to_OG_orientation',unrotated)
return

```

Listing 14: Flame Projection



Figure 21: Projected Booster Flames

6.4.2 Projected Shuttle

Secondly, with the alternative tracking via colors, I wanted a way to map a detailed image of the shuttle onto the tracked object, meaning it could look drastically different than the real model. While I did not use a drastically different image, this serves as a proof of concept for the possibility of projecting anything onto something that need not include a physical model, just a few colors. The shuttle projection takes place in the same portion of the code as flame projection, making use of all the same rotation and opacity techniques mentioned. Unlike the flames, this shuttle uses two values to determine the scale of the image to project atop the center point. The wingspan and length are based on the distance from center to nose and wingtip and compared to the standard mapping, creating an x and y scale respectively for the projected image. Code snippet 15 shows that this is essentially the same action as flame projection, though the alternative tracking method was instrumental in easily defining a length and wingspan. An image with the projected shuttle is shown in Fig. 22.

```
def Flame_Projection():
# Project shuttle
Scaled_shuttle = cv2.resize(shuttle, dsize = (0,0),
                             fx = (ship_wingspan/standard_wingspan), fy = (ship_length/
                             standard_length))
shuttle_size = np.shape(Scaled_shuttle)
print(shuttle_size)
alpha_s = 1 #Scaled_shuttle[:, :, 3] / 255.0
alpha_l = 1.0 - alpha_s
TL = np.array([ anchor_points[4][0]+center[0]-shuttle_size[1]/2 + offset ,
                 anchor_points[4][1]+center[1]]) .astype('int16')
BR = np.add(TL, [shuttle_size[1] + offset , shuttle_size[0]]) .astype('int16')
for c in range(0, 3):
    rotated[TL[1]:BR[1], TL[0]:BR[0], c] =
        (alpha_s * Scaled_shuttle[:, :, c] +
         alpha_l * rotated[TL[1]:BR[1], TL[0]:BR[0], c])
```

Listing 15: Shuttle Projection



Figure 22: Projected Shuttle Image

6.4.3 Shuttle Trail

Lastly, the path the system takes in correcting angle and position might not be easy to observe in detail from the side as a patron of the museum, but is important in understanding how the system behaves. To make this a bit clearer, a trail is created on the screen as the shuttle moves into place. This trail has a buffer of 64 positions, meaning it will plot up to the past 64 center positions, and scales based off of how far back in the trail a point was. This has the effect of creating a fading trail that can show a good deal of where the shuttle has been. Code snippet 16 shows the simple addition of a trail to the image, seen in Fig. 23. An interesting tool used here is the "deque" object in python, which is a double ended queue. When appending a piece of data to the "left" side of the deque, it will take the first index. Placing data into a full deque knocks the last point out and shifts all indices up one to make room for the new data. In this way, the oldest data is lost once you exceed the buffer length, but you can effectively hold onto only as much data as you care about, which is perfect for my application.

```
def Position_Overlay():
    # Draw trail
    Trail.appendleft((center[0], center[1]))
    for jj in xrange(1, len(Trail)):
        if Trail[jj-1] is None or Trail[jj] is None:
            continue # Skips iteration since no line can be drawn
        thickness = int(np.sqrt(Buffer.Length)/ float(jj+1))*2
        cv2.line(image_focused, Trail[jj-1], Trail[jj], [0,0,125], thickness)
```

Listing 16: Trail Projection

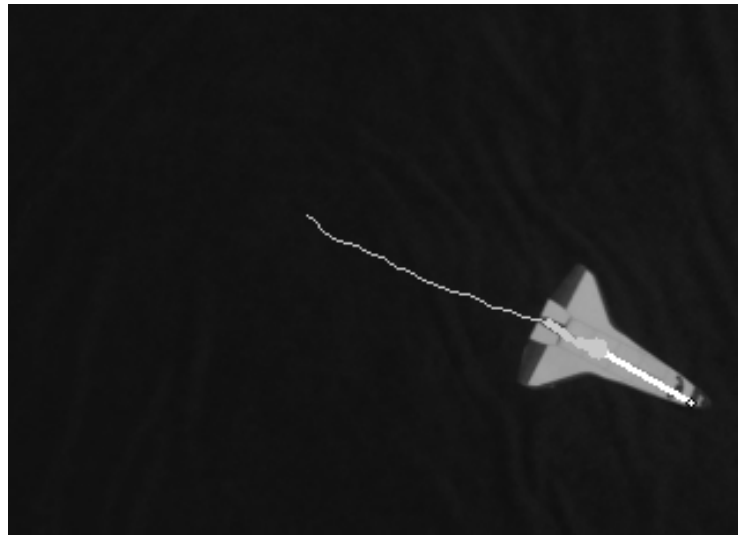


Figure 23: Fading Trail Tracking

7 Control

In this section, the controls as handled aboard the shuttle's Arduino Mega will be discussed, along with the simulations which gave rise to the design of the controller.

7.1 Control Command

One major area for change within the controls will be how the coordinate system is defined, and what the computer vision python code sends to the Arduino Mega. Previously, it appeared that X and Y were defined as the pixel location, which would be a logical way to define that since it would require not matrix transforms. However, the system is controlled by multiple SISO systems sequentially, rather than a simultaneous MIMO system. To elaborate on why the previous method could be problematic, consider the cases shown in Figs. 24 & 25. This shows that if θ is at 45 degrees, then you would end up correcting the angle first, but each of the directional motors moves the system in both x and y since the shuttle is at an angle. This can be corrected if I consider the x and y to be defined perpendicular and parallel respectively to the orientation of the target. This will involve applying some transforms to the detected position to put it into a new coordinate system, but will allow the improvement to behaviour just mentioned, as well as eliminating the need for any math to determine the difference in position and target, since they will be defined from an origin at the target.

Handling this conversion of coordinate systems is relatively simple in python, as it is essentially a 2D rotation and linear translation. First, I normalize the position of the shuttle around the centroid of the target. Next, using the orientation of the target, I rotate the coordinates of shuttle to be relative to the coordinate system defined at the shuttle and add the two orientations to arrive at the relative orientation. Each of these values is then put into a string and sent via Xbee, as discussed in an earlier section. code snippet 17 shows the calculations and communication for the command string. How this string is parsed was discussed in the prior section on the shuttle.

```
Rotation=np.array([[math.cos(theta_Target),-1*math.sin(theta_Target)], ...
                  [math.sin(theta_Target),math.cos(theta_Target)]]);
...
##Define relative coordinates for command line
theta_Relative=int(math.degrees(theta-theta_Target))
position=np.multiply(scale,np.dot(Rotation,[center[0]-Target[0],...
                                         center[1]-Target[1]])).astype(int)
## Scales the relative position in terms of meters
## Writes to the serial port the command line
Command='AT'+str(theta_Relative)+'X'+str(position[1])+'Y'+str(position[0])+'Z'
ser.write(Command) ##Sends via Xbee "ATtheta,XdX,YdYZ"
```

Listing 17: Coordinate System Conversion and Control Communication

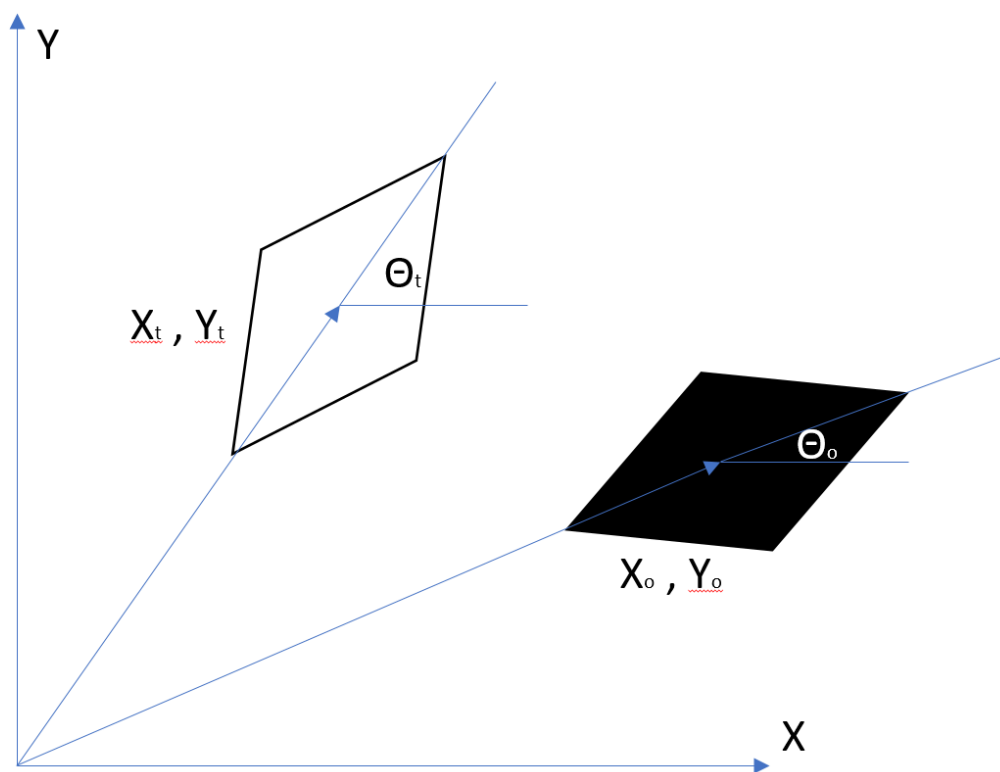


Figure 24: Traditional Coordinate Definitions for Positioning

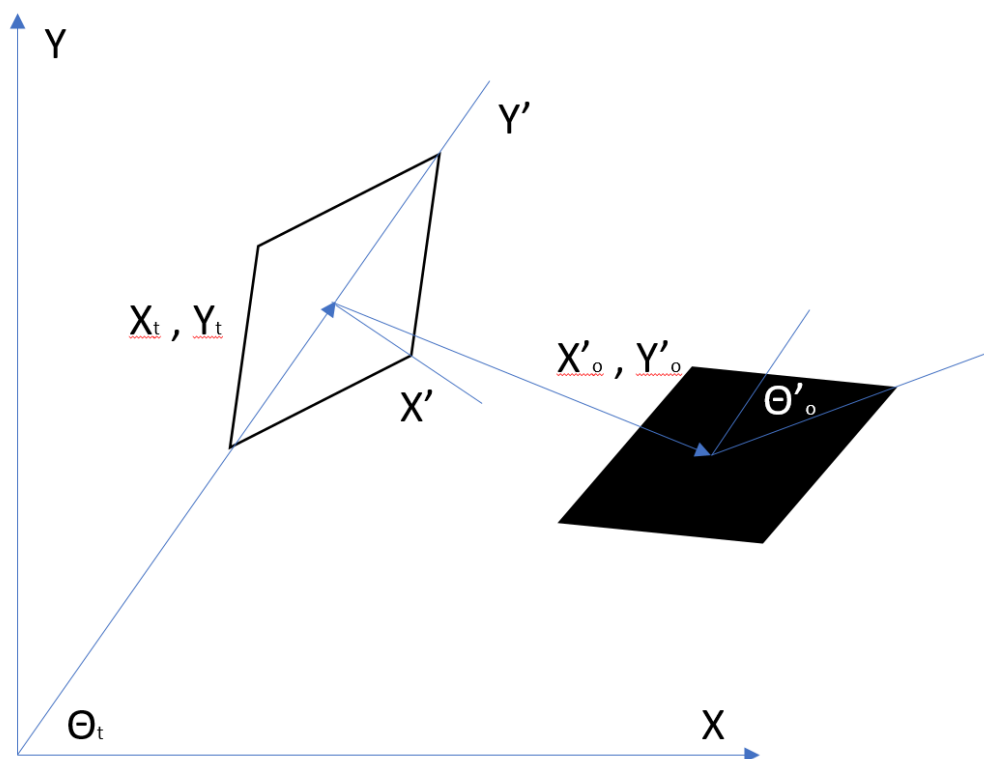


Figure 25: Target-Based Coordinate Definitions for Positioning

7.2 Controller Design

The next major step for this project was creating and implementing the controller into the feedback loop. This is done on the Arduino Mega, which receives the delta command and will output the motor speed settings to pilot the shuttle into place. The system ID which was described in the shuttle portion informed the simulations and ultimately the fine-tuned controller design. After designing a controller for each form of motion, they were discretized and implemented into the Arduino code.

7.2.1 Simulation of Physical Model

Three ODE's and transfer functions were established for each of the movement styles: Axial, Lateral, and Rotational. The equations are very similar, each being a second order system with one pure integrator and two parameters which define the other pole, but for two important distinctions. The lateral movement is provided by two fans simultaneously, while the axial only has the rear or front fan to provide force, and the rotational equation deals in moment of inertia and torque rather than mass and force like the other two. For the linear motion, mass (m) was calculated by simply weighing the overall system (and individual components which were placed accurately on the model), and the moment of inertia (I) calculated via SolidWorks once the model was constructed. The damping parameter (b), arises from a combination of air resistance and sheer force along the bottom edge due to movement along the air table. For our purposes, the speeds will be so low that I made an assumption that shear would significantly overpower the air resistance, and only calculated the damping which would arise from this shear. The coefficients and calculations used to find b are shown below in the Matlab code snippet showing the creation of the model TF of axial movement.

```
u=1.846e-005; %kg/ms - kinematic viscosity
A=.0696; %m^2 - surf area, calculated on SW
y=.005; %m, assumed distance from shuttle to ground
M=.85; %kg - measured
b=A*u/y; % calculated damping parameter
Gy=tf(1,[M b 0]) %Plant for axial movement
C=20*tf([1 .12],[1 15]); %Sample lead controller
L=C*Gy; F=feedback(L,1);
```

The resulting approximate transfer functions for each of the different SISO systems are shown below in Eqns. 2-4. F_y is the force which results from either the front or back fan, F_x is the combined force from both fans on either side of the shuttle, and τ is the torque produced by opposing fans at the front and back of the shuttle when rotating.

$$G_Y(s) = \frac{Y}{F_Y} = \frac{1}{ms^2 + bs} = \frac{1}{0.85s^2 + 0.000257s} \quad (2)$$

$$G_X(s) = \frac{X}{F_{X1} + F_{X2}} = \frac{X}{2 * F_X} = \frac{1}{ms^2 + bs} = \frac{1}{0.85s^2 + 0.000257s} \quad (3)$$

$$G_\theta(s) = \frac{\theta}{\tau} = \frac{\theta}{\tau_1 + \tau_2} = \frac{1}{Is^2 + bs} = \frac{1}{0.003897s^2 + 0.000257s} \quad (4)$$

The plot of the step response with feedback control and a lead compensator is shown in 26. While this response looks very promising, it does not take into account a critical factor of the system. Our thrusters have a very low ceiling of force output, and thus saturate very quickly. This is not

necessarily a bad thing, but does impact the model and performance significantly. Additionally, thrusters being active at full force or not at all is a method of control known as "bang-bang" which is actually used for efficiency purposes in spaceflight, making this model all the more realistic.

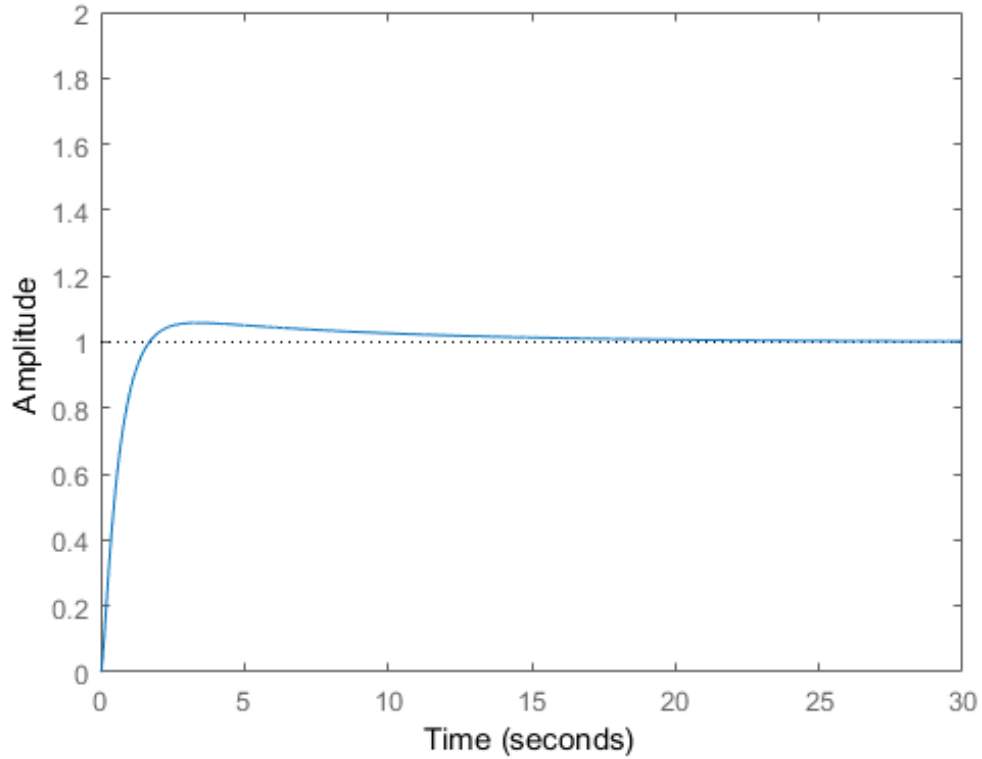


Figure 26: Ideal Step Response of System in Feedback

Since our ideal controller does not reflect the saturating reality of our actuators, I instead modeled the system in Simulink, integrating the voltage-force relationships derived for our fans earlier and the voltage limits imposed by the motor controller. One of these simulations is shown below in fig 27. Note the logic, which decides which fan or set of fans will receive a command, which is integrated into the final control code aboard the shuttle. Additionally, Figs 28 and 29 show graphs generated by this simulation are shown to explain the effect of saturation and the controller design. These graphs simulate the autopilot being activated while the shuttle is 1 m 'below' the target in the Y' direction.

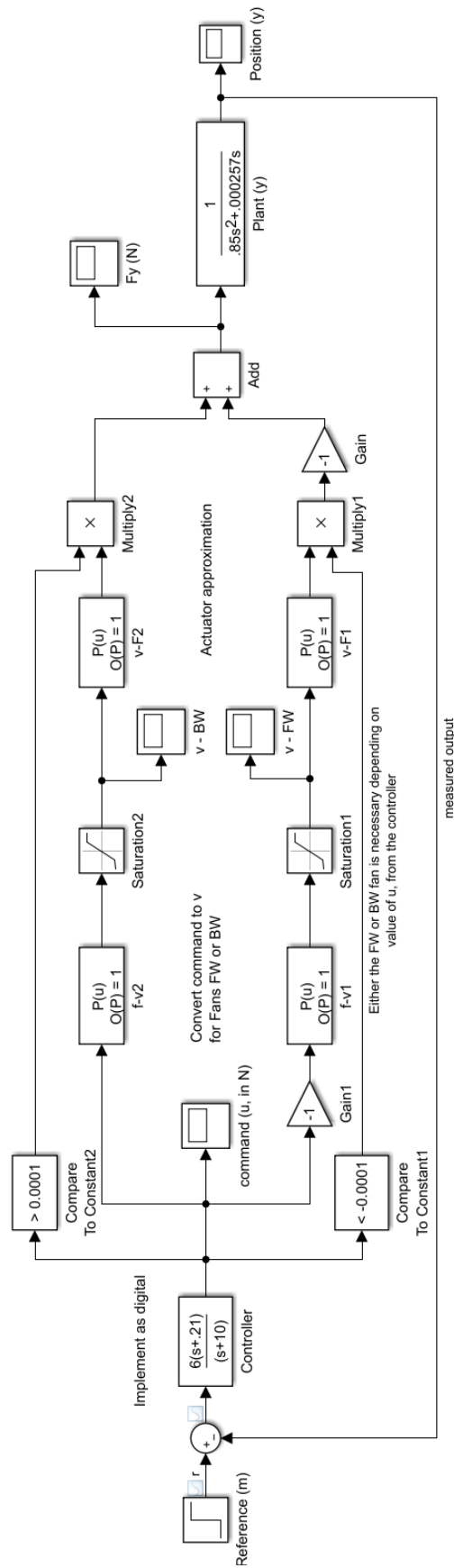


Figure 27: Simulink Model for Axial Position Control

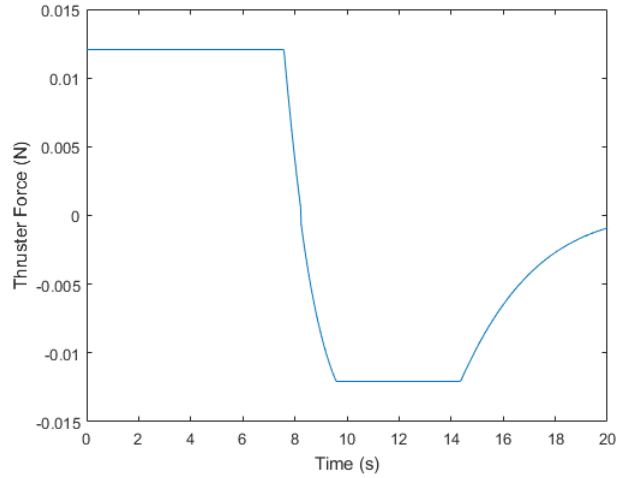


Figure 28: Simulated Force Response to a 1 m Step

It is clear from the graph of the force response that the fans will be saturated the majority of the time the system is in autopilot. Despite this non-linearity, the controller is designed such that we still get a relatively quick and smooth response, as seen below.

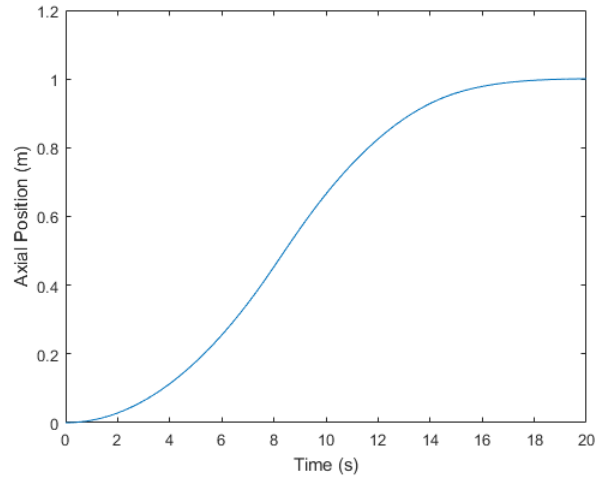


Figure 29: Simulated Response To a 1 m Step

Once the simulation was created, the specific controllers for each system were developed through a combination of root locus and evaluation with various parameters. Though they are all lead controllers with relatively close poles and zeros, the locations of those poles and zeros is critical to the optimal behavior which was found after much experimentation. Each controller was made to result in a response with no oscillation, somewhat smooth force dissipation as the target was approached, and the fastest rise time possible. The current controllers are shown below in Eqns. 5-7.

$$C_Y(s) = \frac{U_Y}{e_Y} = \frac{K(s + P_Y)}{(s + Z_Y)} = \frac{6(s + 0.21)}{(s + 10)} \quad (5)$$

$$C_X(s) = \frac{U - X}{e_X} = \frac{K(s + P_X)}{(s + Z_X)} = \frac{3(s + 0.315)}{(s + 5)} \quad (6)$$

$$C_\theta(s) = \frac{U_\theta}{e_\theta} = \frac{K(s + P_\theta)}{(s + Z_\theta)} = \frac{0.5(s + 0.25)}{(s + 15)} \quad (7)$$

As mentioned in the data collection sub-section, this model will be validated in testing by saving tracking data and comparing the results to simulated responses from the models. This may change the controllers when the parameters are updated, though the improvement is unlikely to be too significant. This will be discussed further in the next iteration of the report after this analysis has been conducted.

7.2.2 Discretization of Controllers

After simulating the different systems and determining the ideal controller for each to optimize response time, those controllers needed to be converted to a usable form for control in a digital and discrete time microcontroller. The method used is the Tustin transform I learned in the course "Industrial Automation," the substitution equation for which is shown below in Eqn. 8. This method converts the transfer function in terms of s to one in terms of Z , depending on the sampling rate. Note that this " Z " is not the same zero referred to in the controller equations, and is essentially a dummy variable which will shortly be irrelevant.

$$s = \left(\frac{2}{T_s}\right)\left(\frac{Z - 1}{Z + 1}\right) \quad (8)$$

The Z transfer function, after being placed in the form Eqn. 9, can then be implemented when put in the form described by 11, which is specifically for controllers with both a singular pole and zero, since there are only two of each coefficient a and b . Note that K is the controller gain, and k refers to the loop number, so that the command depends on the error and command from the prior loop.

$$U[k] = \frac{b_0 + b_1 Z^{-1} \dots b_M Z^{-M}}{a_0 + a_1 Z^{-1} \dots a_N Z^{-N}} \quad (9)$$

$$U[k] = \frac{b_0 e[k] + b_1 e[k - 1] - a_1 U[k - 1]}{a_0} \quad (10)$$

$$U[k] = \frac{T_s K Z + 2K}{PT_s + 2} e[k] + \frac{T_s K Z - 2K}{PT_s + 2} e[k - 1] - \frac{PT_s - 1}{PT_s + 2} U[k - 1] \quad (11)$$

The above is the form of the controller which the Arduino uses, calculating the coefficients for each system on initialization, and using them to calculate the command U each loop. The sampling time, T_s , used in this case is the conveniently matching speed of both the Arduino and Python code, 0.2 s or 50 Hz, and $K, P, \&Z$ are the gain, pole, and zero for each system. These coefficients may change depending on the model changes, but are implemented in the Arduino code in such a way that this will be a simple change to the code described in the next section.

7.3 Implementation

With the controllers designed and the framework in place to convert those transfer functions to discrete time, codified controls, the next step was to implement the control calculations and commands into the python code. Firstly, the discretization equation (Eqn. 11) was implemented in the initialization code and the main loop as shown in code snippet 18.

```
m = 1500
b = 49
Z = [0.25, 0.315, 0.21]
P = [15.0, 5.0, 10.0]
K = [0.5, 3.0, 6.0]
dT = 0.02
theta_old = 0
position_old = [0, 0]
theta_threshold = 10
x_threshold = 20
y_threshold = 20
u = [0, 0, 0]
u_old = [0, 0, 0]

...
# In Main loop, calculate coeffs for control
for ii in range(0, 3):
    a0[ii] = P[ii] * dT + 2
    c1[ii] = (dT * K[ii] * Z[ii] + 2.0 * K[ii]) / a0[ii]
    c2[ii] = (dT * K[ii] * Z[ii] - 2.0 * K[ii]) / a0[ii]
    c3[ii] = (dT * P[ii] - 1.0) / a0[ii]
```

Listing 18: Digital Controller Initialization

Depending on if the position values are within specified thresholds, 5 degrees and .1 m, the controller will then use the calculated command to determine which fans to activate and at what power. Theta takes precedence, and must be corrected to within 5 degrees before the axial or lateral positions are controlled. Once it has been corrected, we begin moving the shuttle into position on the target. The code for deciding the motor speed depending on each command is shown in code snippet 19.


```

# If theta not resolved yet
v = [0, 0, 0, 0, 0, 0]
if abs(theta_Relative) > theta_threshold:
    u[0] = c1[0] * theta_Relative + c2[0] * theta_old - c3[0] * u_old[0]
    if u[0] >= 0:
        v[0] = m*u[0] + b
        v[3] = v[0]
    elif u[0] <= 0:
        v[1] = -m*u[0] + b
        v[2] = v[1]
else: #abs(position[0]) > x_threshold or abs(position[1]) > y_threshold:
    u[0] = c1[0] * theta_Relative + c2[0] * theta_old - c3[0] * u_old[0]
    # Position commands use scaled position to get values in meters
    u[1] = c1[1] * position[0] + c2[1] * position[0] - c3[1] * u_old[1]
    u[2] = c1[2] * position[1] + c2[2] * position[1] - c3[2] * u_old[2]
    if u[0] <= 0:
        v[0] = -m*u[0]
        v[3] = v[0]
    elif u[0] > 0:
        v[1] = m*u[0]
        v[2] = v[1]
    if u[1] < 0:
        v[0] = -m*u[1] + b + v[0]
        v[2] = v[0] + v[2]
    elif u[1] > 0:
        v[1] = m*u[0] + b + v[1]
        v[3] = v[1] + v[3]
    if u[2] < 0:
        v[4] = -m*u[2] + b
    elif u[2] > 0:
        v[5] = m*u[2] + b
for ii in range(0, 6):
    if v[ii] > 255 or v[ii] < 0:
        v[ii] = 255
    elif v[ii] > 0 and v[ii] < b:
        v[ii] = b
# Writes to the serial port the command line of motor speeds
Command = 'A' + str(int(v[0])) + 'B' + str(int(v[1])) +
          'C' + str(int(v[2])) + 'D' + str(int(v[3])) +
          'E' + str(int(v[4])) + 'F' + str(int(v[5])) + 'Z'
ser.write(Command)
print ('Send to Shuttle:.. ' + Command)
u_old = u
position_old = position # Store position for next calculation
theta_old = theta_Relative

```

Listing 19: Digital Controller For all Three States Concurrently

The logic statements regarding sign of the command for each system make sure that only one set of fans for each system will be active, though it may be rotating and moving sideways, which will result in some amount of wasted power, but this should be relatively minimal. Once the delta values are within the thresholds, the system will be considered docked and will not power the fans. The logic and corrective fan powering has been confirmed through simulated tests and the true efficacy of this controller will be tested at the exhibit within the next month, alongside data collection, and it will be adjusted as necessary.

8 Testing

8.1 Test Procedure

In order to expedite the process of testing, especially in the case of an onsite DMNS test, but also for a test on the Aerospace Department's air table, the following steps lay out the process of testing the system and establishing data for system identification. **Setup:**

1. Bring required equipment:
 - (a) Joystick Controller & Shuttle
 - (b) Laptop with charging cable and serial connector for Arduino
 - (c) Raspberry Pi with camera, monitor, keyboard, mouse, and power supply
 - (d) Extension Cable
 - (e) Tape and Lego to mount camera
2. Attach Lego to ceiling, and camera case to lego
3. Plug in Raspberry Pi
4. Place Shuttle (with shuttle cutout) on surface, check battery, flip power switch
5. Turn on controller
6. Test each direction of joystick, confirming shuttle movement
7. Run `Camera.Setup.py`, move shuttle to limits to confirm scale factor (portion) and note it
8. Change portion in each script to newly found value: data collectors, and position det
9. If threshold screen shows additional objects on screen, adjust TH value higher until only the shuttle/target are visible. Change in all scripts, and note that value.

System ID and Data Collection:

1. Place parking space cutout on field
2. Place shuttle in FOV
3. Set controller toggle to A(utomatic)
4. Run *Data_Collector_Rotate.py*
5. Confirm that target has white line inlaid showing orientation, and that shuttle has black lines. If these are not correctly ordered, swap the cnt values within script (do so for all scripts)
6. Repeat 4 at least 2 more times, changing the fname variable to save to a dif file
7. Place shuttle on right edge of FOV
8. Run *Data_Collector_Left.py* at least 3 times
9. Place shuttle on Bottom edge of FOV
10. Run *Data_Collector_Up.py* at least 3 times

Controls / Autopilot:

1. Place Shuttle away from Target
2. Switch controller toggle to Automatic)
3. Run *PositionDetection_Video_Control_EdgeDetector.py*
4. Pray it automatically seeks the target
5. Evaluate collected data and observed behavior to adjust control values

8.2 Data Collection

In order to validate the model I have created, data will be collected from a test which will occur over the following month. This test will validate the control schema, and be used to reverse engineer a model for the system, giving a different approach for system ID than has so far been used. This data will be created in 4 tests, multiple occurrences of will be run. The first test will be of the overall control scheme, discussed in the following section, and the next 3 will be simplified tests which run the fans in one direction for a set amount of time. These will validate the models for thruster forces, as well as provide a more accurate b value for the model, which may allow for improved performance from the controller once integrated. Code snippet 20 shows how python will hold onto data values within the loop for the duration of tests and then write that data to a file at the end. This data is processed by the code in Appendix D, which calculates the damping parameter $\mathbf{b}$, to help improve the model of the system.

```
def Save_Data():
    #Save data as (rel X, rel y, rel Theta, dT, abs x, abs y, abs theta)
    Data[i]=[position[0], position[1], theta_Relative, dT, ...
            center[1], center[0], int(math.degrees(theta))]
    i=i+1
    ...
DataFname="/home/pi/Desktop/AutoPilot/CV/Data_Collection_Sideways.csv"
np.savetxt(DataFname, Data, delimiter=",")
```

Listing 20: Data Collection

8.3 Results Discussion

So far, testing has been minimal since the only available testing bed is a small, imbalanced, and poorly lit air table. While this air table has been incredibly helpful in ironing out kinks in communication and controls, a full-scale test simply cannot be reliably conducted. A DMNS visit will take place shortly to allow the full procedure outlined above, whereupon any final improvements for the controls may be recommended.

9 Conclusion

The purpose of this project was threefold, and served as a sort of capstone for my MS by tying my entire schooling together. The primary goal was to apply the knowledge gained in the course on Computer Vision to a more real-world project. Secondly, I wanted to implement a digital controller, tying together many courses in an application so as to reinforce the concepts of Industrial Automation and Feedback Control. Lastly, it provided an excellent exercise in integration of systems, having 3 separate systems connected by a variety of communications methods and across two coding languages. In each of those major fields, I felt that I accomplished a significant amount, faced many novel challenges, and grew significantly as an engineer.

The Computer Vision aspect of this project was what initially excited me the most, and has provided some of the most valuable lessons. As such a unique process, there were a surprising number of tricks that I picked up and implemented along the way to create a viable CV system. As of the end of the Fall semester, I was operating at roughly 30 Hz, and have since increased that to 80 Hz with increased effectiveness and robustness. While speed is not the most important factor, especially on a slow system like this shuttle, it is a good measure of how greatly my finesse with OpenCV and Python has improved. In speeding processing up, I also delved deeper into some of the more arcane functions of OpenCV to recreate their effectiveness and better understand the tools I was using. Additionally, working more with Augmented Reality concepts, as covered in section 6.4, opened up a lot of ideas about how to keep playing with CV going forward.

The controls implementation turned out to be far more difficult to get right than I anticipated, showing that my theoretical background in feedback control design was insufficient without some hands-on experience. Now that I have successfully implemented the controller, I feel far more confident in my ability to do so in future projects. This particular controller was made quite interesting by the non-linearity caused by the saturation of the fans on the shuttle. These fans are extreme examples of much a saturated actuator can effect controls, and they led me to use the "Bang-Bang" approach to control which is a valuable lesson for someone going into the aerospace industry. The rest of the system integration went fairly smoothly, and gave some valuable lessons on communication between devices.

While there were an immense number of valuable lessons and great leaps in my understanding of system control, there were also some things I wish I could have done better. Primarily, the opportunity to test at the museum arriving earlier may have helped me see issues in the controls and detection earlier and make more improvements. Luckily, the air table I was able to test on did help catch some big issues, but did not let me test the system as fully as I would have liked. The PiCam is a valuable tool to use with the Raspberry Pi, but there are other options that might have been valuable to explore, even if I didn't fully replace the PiCam with them. These are fairly small what-ifs compared to the knowledge I did gain in this truly enriching project.

Below is the complete code along with comments for all programs. It may provide some additional context for my comments above, however the critical segments have been highlighted throughout the report.

A Joystick Controller Arduino Uno

```
1 void setup() {
2   // put your setup code here, to run once:
3   Serial.begin(115200);
4   //Begins the serial port using TX and RX at Baud rate for Xbee
5   pinMode(2, INPUT); //set up pins to receive input from joysticks
6   pinMode(3, INPUT);
7   pinMode(4, INPUT);
8   pinMode(5, INPUT);
9   pinMode(6, INPUT);
10  pinMode(7, INPUT);
11  pinMode(8, INPUT);}
12 void loop() {
13   if (digitalRead(8) == HIGH) { //If the Toggle is in the Down "Manual
14     " position
15     //Series of if statements to determine which direction joystick is
16     pressed
17     if (digitalRead(2) == HIGH) {
18       // Serial.println("Move Left");
19       Serial.println("L");    }
20     if (digitalRead(3) == HIGH) {
21       // Serial.println("Move Down");
22       Serial.println("D");    }
23     if (digitalRead(4) == HIGH) {
24       // Serial.println("Move Right");
25       Serial.println("R");    }
26     if (digitalRead(5) == HIGH) {
27       // Serial.println("Move Up");
28       Serial.println("U");    }
29     if (digitalRead(6) == HIGH) {
30       // Serial.println("Turn Right");
31       Serial.println("Q");    }
32     if (digitalRead(7) == HIGH) {
33       // Serial.println("Turn Left");
34       Serial.println("E");    }
35   } else {
36     //If the toggle is up "Autopilot" then send X, indicating..
37     that instructions should be taken from the other Xbee
38     Serial.println("X");    }}
```

B Shuttle Arduino Mega

```
1 #include <Wire.h>
2 #include <Adafruit_MotorShield.h>
3
4 // put your setup code here, to run once:
5
6 //#include "utility/Adafruit_PWMServoDriver.h"
7 // Establishes the motor controls using the 2 motor shields
8 Adafruit_MotorShield AFMSbottom = Adafruit_MotorShield(0x60);
9 Adafruit_DCMotor *RightFront = AFMSbottom.getMotor(1);
10 Adafruit_DCMotor *LeftFront = AFMSbottom.getMotor(2);
11 Adafruit_DCMotor *LeftBack = AFMSbottom.getMotor(3);
12 Adafruit_DCMotor *RightBack = AFMSbottom.getMotor(4);
13 Adafruit_MotorShield AFMStop = Adafruit_MotorShield(0x61);
14 Adafruit_DCMotor *Bwd = AFMStop.getMotor(1);
15 Adafruit_DCMotor *Fwd = AFMStop.getMotor(3);
16 char val; // Data received from the serial port
17 String ReadDataOld = "A100B50C200D0E0F0Z";
18 float t0;
19 int v[] = {0, 0, 0, 0, 0, 0}; //Motor speeds initialized at 0
20
21 void setup() {
22     AFMStop.begin(); //Starts the connection with the motor shields
23     AFMSbottom.begin();
24     Serial2.begin(115200); //Starts the serial connections with Xbees on
        sets of TX and RX pins
25     Serial1.begin(115200);
26     Serial.begin(115200); //Starts communication over serial port for
        messages to be read on computer
27 }
28
29 void loop() {
30     t0=millis();
31     // Reset the data strings
32     String ReadData = "";
33     String Control_string = "";
34     char dir;
35     if (Serial2.available()) { //If there is a message from the
        controller Xbee
36         dir = Serial2.read();
37         if (dir == 'R') { //If the controller sent the signal 'R'
38             RightBack->setSpeed(0);
39             Fwd->setSpeed(0);
40             Bwd->setSpeed(0);
41             RightFront->setSpeed(0);
42             LeftFront->setSpeed(255);
```

```

43     LeftFront->run(FORWARD); //Run the motor forward at the set
        speed
44     LeftBack->setSpeed(255); //Set motor 3 (on the right) to full
        power
45     LeftBack->run(FORWARD); //Run the motor forward at the set speed
46     Serial.println("Controller_Right"); //Print to the terminal what
        control was received
47 }
48 if (dir == 'L') {
49     LeftBack->setSpeed(0);
50     Fwd->setSpeed(0);
51     Bwd->setSpeed(0);
52     LeftFront->setSpeed(0);
53     RightFront->setSpeed(255);
54     RightFront->run(FORWARD);
55     RightBack->setSpeed(255);
56     RightBack->run(FORWARD);
57     Serial.println("Controller_Left");
58 }
59 if (dir == 'U') {
60     RightBack->setSpeed(0);
61     LeftBack->setSpeed(0);
62     Bwd->setSpeed(0);
63     RightFront->setSpeed(0);
64     LeftFront->setSpeed(0);
65     Fwd->setSpeed(255);
66     Fwd->run(FORWARD);
67     Serial.println("Controller_Forward");
68 }
69 if (dir == 'D') {
70     RightBack->setSpeed(0);
71     Fwd->setSpeed(0);
72     LeftBack->setSpeed(0);
73     RightFront->setSpeed(0);
74     LeftFront->setSpeed(0);
75     Bwd->setSpeed(255);
76     Bwd->run(FORWARD);
77     Serial.println("Controller_Backward");
78 }
79 if (dir == 'Q') {
80     RightBack->setSpeed(0);
81     Fwd->setSpeed(0);
82     Bwd->setSpeed(0);
83     LeftFront->setSpeed(0);
84     LeftBack->setSpeed(255);
85     LeftBack->run(FORWARD);
86     RightFront->setSpeed(255);

```

```

87     RightFront->run(FORWARD);
88     Serial.println("Controller_Turning_Left");
89 }
90 if (dir == 'E') {
91     Fwd->setSpeed(0);
92     Bwd->setSpeed(0);
93     LeftBack->setSpeed(0);
94     RightFront->setSpeed(0);
95     RightBack->setSpeed(255);
96     RightBack->run(FORWARD);
97     LeftFront->setSpeed(255);
98     LeftFront->run(FORWARD);
99     Serial.println("Controller_Turning_Right");
100 }
101 if (dir == 'O') {
102     RightBack->setSpeed(0);
103     Fwd->setSpeed(0);
104     Bwd->setSpeed(0);
105     LeftBack->setSpeed(0);
106     RightFront->setSpeed(0);
107     LeftFront->setSpeed(0);
108     Serial.println("Controller_Not_Engaged");
109 }
110 if (dir == 'X') {
111     //If the toggle is set to auto-pilot control
112     Serial.println("Computer_Time"); // Print to terminal that the
        auto-pilot is taking over
113     if (Serial1.available() > 0) { // If there is data from the Pi
        Xbee
114         val = Serial1.read();
115         Serial.print(val);
116         if (val == 'A') { //Initiator symbol for a new command
117             for (int i = 0; i < 50; ++i) { //Read in the 25 characters
                of the string
118                 val = Serial1.read();
119                 if (val == 'Z') { //If the end symbol '>' is read, end the
                    loop
120                     ReadData = String(ReadData + val);
121                     Serial.print("Read_Data:");
122                     Serial.println(ReadData); // Print the full string to
                        the console
123                     break;
124                 }
125             else {
126                 ReadData = String(ReadData + val); // Concatenate the
                    new character to the end of the string
127             }

```



```

128     }
129     ReadDataOld = ReadData; //Save the newly read data
130 }
131 }
132 else {
133     ReadData = ReadDataOld; //If no new data, then use the same
        data
134 }
135
136 //With Data read in, set the delta values from the string
137 for (int i = 0; i < 50; ++i) { //Go through the data string
    until
138     if (ReadData[i] == 'A' ) { //If the start character is read
139         Control_string = "";
140         ++i; //Skip the T character to reach the actual Theta number
141     }
142     else if (ReadData[i] == 'B') { // If b character is read,
        convert current string to v[0]
143         v[0] = Control_string.toInt();
144         Control_string = "";
145         ++i; //Skip the X character to start reading the X value
146     }
147     else if (ReadData[i] == 'C') { // If c character is read,
        convert current string
148         v[1] = Control_string.toInt();
149         Control_string = "";
150         ++i;
151     }
152     else if (ReadData[i] == 'D') { // If d character is read,
        convert current string
153         v[2] = Control_string.toInt();
154         Control_string = "";
155         ++i;
156     }
157     else if (ReadData[i] == 'E') { // If e character is read,
        convert current string
158         v[3] = Control_string.toInt();
159         Control_string = "";
160         ++i;
161     }
162     else if (ReadData[i] == 'F') { // If d character is read,
        convert current string
163         v[4] = Control_string.toInt();
164         Control_string = "";
165         ++i;
166     }

```

```

167     else if (ReadData[i] == 'Z') { // If z character is read,
        convert current string
168         v[5] = Control_string.toInt();
169         Control_string = "";
170         ++i;
171         break; //Exit this loop as data has been extracted from the
            string
172     }
173     Control_string = String(Control_string + ReadData[i]); // Add
        the characters to the string
174 }
175 Serial.print("v_=_");
176 for (int i = 0; i < 5; ++i) {
177     Serial.print(v[i]); //Print the motor speeds set by the
        control code
178     Serial.print(",_");
179 }
180 Serial.print(v[5]);
181 Serial.println("_");
182 RightFront->setSpeed(v[0]);
183 RightFront->run(FORWARD);
184 LeftFront->setSpeed(v[1]);
185 LeftFront->run(FORWARD);
186 RightBack->setSpeed(v[2]);
187 RightBack->run(FORWARD);
188 LeftBack->setSpeed(v[3]);
189 LeftBack->run(FORWARD);
190 Fwd->setSpeed(v[4]);
191 Fwd->run(FORWARD);
192 Bwd->setSpeed(v[5]);
193 Bwd->run(FORWARD);
194 }
195 }
196
197 else { //With no commands from the Controller Xbee, turn off fans
198     RightBack->setSpeed(0);
199     Fwd->setSpeed(0);
200     Bwd->setSpeed(0);
201     LeftBack->setSpeed(0);
202     RightFront->setSpeed(0);
203     LeftFront->setSpeed(0);
204 }
205 Serial.print("Elapsed_Time_(ms)=_"); Serial.println(millis()-t0);
206 }

```

C Python Computer Vision

The Code for CV is broken up into functions which are called from the main loop. Setup() and Target_Detection() are run once on initialization, and then the main loop runs until data has been collected or the quit command is given.

C.1 Main Loop

```
def main():
    global dT, t1, ii
    # ----- Loop to identify object and send delta command -----
    for ii in range(0, np.shape(Data)[0]): # Run for amount of data desired
        try:
            dT = time.clock() - t1
            t1 = time.clock()
            Image_Read() # Read Image From Capture Object
            Image_Processing() # Thresholding and Morphology
            ContourDetection_findContours() # Find center and Tip
            Position_Calc()
            Command_Calc()
            # Optional Informational Displays
            Flame_Projection()
            Position_Overlay()
            Save_Data()
        except:
            print("Something_Broke!")
    # if the 'q' key was pressed, break from the loop
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
    return
```

C.2 Setup

```
def setup():
    global Data, DataFname
    # Data saving
    Data = np.zeros((1500, 16))
    # ALTER FNAME EACH TEST!
    DataFname = "Data_Collection_Controlled_1.csv"

    # Declaring values used in the main loop
    global kernel2, kernel3, kernel4, kernel5
    kernel2 = np.ones((2, 2))
    kernel3 = np.ones((3, 3)) # Used for closing operation
    kernel4 = np.ones((4, 4))
    kernel5 = np.ones((5, 5))

    # Declaring control values
    global m, b, Z, P, K
    m = 160
    b = 45
    Z = [0.25, 0.315, 0.21]
    P = [15.0, 5.0, 10.0]
```

```

K = [0.5, 3.0, 6.0]
global u, a0, c1, c2, c3, dT, t1
u = [0, 0, 0]
v = [0, 0, 0, 0, 0, 0]
a0 = [0, 0, 0]
c1 = [0, 0, 0]
c2 = [0, 0, 0]
c3 = [0, 0, 0]
dT = 0.015
t1 = time.clock()
global u_old, theta_old, position_old, theta_threshold
global x_threshold, y_threshold
theta_old = 0
position_old = [0, 0]
u_old = [0, 0, 0]
theta_threshold = 10
x_threshold = 20
y_threshold = 20

# Open serial port
global ser
PORT = '/dev/ttyUSB0' # Sets the serial port as the USB port
BAUDRATE = 115200 # Baud rate which Xbee is configured to read/send data
ser = serial.Serial(PORT, BAUDRATE)

# Defining the size of the image the camera will take and camera settings
global rows, cols, imagefocus, portion, offset
rows = 240
cols = 320
# Define the range of the image which we care about
portion = 4
# Define the portion as 2/'portion' of img (i.e. center half is portion=4)
imagefocus = np.array(
    [[cols / 2 - cols / portion, cols / 2 + cols / portion],
     [rows / 2 - rows / portion, rows / 2 + rows / portion]])

# Camera Settings
global cap
#cap = PiVideoStream(resolution = (cols,rows), framerate = 60).start()
cap = cv2.VideoCapture(0)
ret = cap.set(cv2.CAP_PROP_FRAME_WIDTH, cols)
print(ret)
ret = cap.set(cv2.CAP_PROP_FRAME_HEIGHT, rows)
print(ret)

# Tracking initialization
global Buffer_Length, Trail
Buffer_Length = 64
Trail = deque(maxlen = Buffer_Length)

# Import Flame and set alphas, then create rotated versions
global jet_scale, Flame_front, Flame_rear, Flame_left, Flame_right
jet_scale = 10
Flame_imported = cv2.imread("Flame.png", -1)
Flame = cv2.resize(Flame_imported, dsize = (cols, rows))
non_white_cells = np.where(Flame != [255, 255, 255, 0])
Flame[non_white_cells[0]][non_white_cells[1]][3] = 255

```

```

Flame_front = Flame
M = cv2.getRotationMatrix2D((cols/2,rows/2),180,1)
Flame_rear = cv2.warpAffine(Flame,M,(cols,rows))
M = cv2.getRotationMatrix2D((cols/2,rows/2),90,1)
Flame_left = cv2.warpAffine(Flame,M,(cols,rows))
M = cv2.getRotationMatrix2D((cols/2,rows/2),270,1)
Flame_right = cv2.warpAffine(Flame,M,(cols,rows))

# Shuttle Projection
global shuttle, real_length, real_wingspan
Shuttle_imported = cv2.imread("shuttle.png",-1)
print(np.shape(Shuttle_imported))
shuttle = cv2.resize(Shuttle_imported[75:-275,125:-25], dsize = (np.shape(
    Shuttle_imported)[1]/20,np.shape(Shuttle_imported)[0]/20))
shuttle = np.array(np.append(shuttle,np.zeros((np.shape(shuttle)[0],np.shape
    (shuttle)[1],1)), axis = 2))
white_cells = np.where(shuttle == [255, 255, 255, 0])
shuttle[white_cells[0]][white_cells[1]][3] = 255
real_length = 0.2032 # 8 inches in meters, rough length of shuttle
real_wingspan = 0.08

#location of jet anchors in x,y at standard length (26.45)
global standard_length, standard_wingspan, Anchors
standard_length = 26.45
standard_wingspan = (standard_length / real_length) * real_wingspan
Right_rear = [-15,-5]
Right_front = [-5,15]
Left_rear = [-Right_rear[0],Right_rear[1]]
Left_front = [-Right_front[0],Right_front[1]]
Front = [0,25]
Rear = [0,-25]
Anchors = np.array([Right_front,Left_front,Right_rear,Left_rear,Rear,Front])

# Declare HSV marker values
global Cyan_MIN, Cyan_MAX, Magenta_MIN, Magenta_MAX, Yellow_MIN, Yellow_MAX
# Cyan - tip
Cyan_MIN = np.array([80, 70, 50],np.uint8)
Cyan_MAX = np.array([130, 255, 255],np.uint8)
# Yellow - center
Yellow_MIN = np.array([20, 50, 50],np.uint8)
Yellow_MAX = np.array([60, 255, 255],np.uint8)
# Magenta - wingtip
Magenta_MIN = np.array([140, 30, 40],np.uint8)
Magenta_MAX = np.array([180, 255, 255],np.uint8)

global center, tip, wing
center = [25,20]
tip = [25,25]
wing = [20,25]

# Undistortion data
global mtx, dist
mtx = np.load('CameraCalibrationMatrix.npy')
dist = np.load('CameraDistortionValues.npy')
time.sleep(1.0)
return

```

C.3 Target Detection

```
def Target_Detection():
    global Target, tip_Target, theta_Target, Rotation
    # ----- Use detection once to find the parking spot -----
    ret, frame_o = cap.read()
    image = frame_o.reshape((rows, cols, 3))
    # Converts img data to grayscale, Cuts image down to focus area
    imageGray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    img = imageGray[imagefocus[1, 0]:imagefocus[1, 1],
                    #imagefocus[0, 0]:imagefocus[0, 1]]

    # Threshold the image
    TH, imgTH = cv2.threshold(img, 100, 255, cv2.THRESH_BINARY)
    # Closing the image to complete the shape of the shuttle
    closed = cv2.morphologyEx(imgTH, cv2.MORPH_ERODE, kernel5, iterations=1)
    #cv2.imshow("Objects", closed)
    # Find the Contours of the img, then specifically calling out the longest
    # one
    im_dummy, contours, hierarchy = \
        cv2.findContours(closed, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    cnt = contours[np.argmax(len(contours[:]))]
    # Moment and centroid definition
    M = cv2.moments(cnt)
    Target = [int(M['m10'] / M['m00']), int(M['m01'] / M['m00'])] # y, x
    # Finds the avg of the contour, defines tip as furthest from the avg.
    # This calc is only usefull because of shape of ship
    dif = np.subtract(cnt[:, Target[:]], Target[:])
    normal = np.sqrt(np.add(np.multiply(dif[:, 0], dif[:, 0]),
                                     np.multiply(dif[:, 0], dif[:, 1]),
                                     np.multiply(dif[:, 1], dif[:, 1])))
    tip_Target = cnt[np.argmax(normal)][0]
    # Angle calculation based on slopes of line
    dy = -Target[0] + tip_Target[0]
    dx = -Target[1] + tip_Target[1]
    theta_Target = math.degrees(math.atan2(dy, dx))
    Rotation = np.array(
        [[math.cos(theta_Target), -math.sin(theta_Target)],
         [math.sin(theta_Target), math.cos(theta_Target)]]
    )
    return
```

C.4 Image Reading

```
def Image_Read():
    global img, image, imageHSV, image_focused
    ret, frame = cap.read()
    img = frame.reshape((rows, cols, 3))
    image_undistorted = cv2.undistort(img, mtx, dist, None)
    image_focused = image_undistorted[imagefocus[1, 0]:imagefocus[1, 1],
                                     imagefocus[0, 0]:imagefocus[0, 1]]
    #cv2.imshow('Focused', image_focused)
    # Converts img data to HSV
    imageHSV = cv2.cvtColor(image_focused, cv2.COLOR_BGR2HSV)
    return
```

C.5 Image Processing

```
def Image_Processing():
    global Cyan_threshed, Yellow_threshed, Magenta_threshed
    # Threshold the image
    Cyan_threshed = cv2.inRange(imageHSV, Cyan_MIN, Cyan_MAX)
    Yellow_threshed = cv2.inRange(imageHSV, Yellow_MIN, Yellow_MAX)
    Magenta_threshed = cv2.inRange(imageHSV, Magenta_MIN, Magenta_MAX)
    cv2.imshow('Cyan', Cyan_threshed)
    cv2.imshow('YLW', Yellow_threshed)
    cv2.imshow('Magenta', Magenta_threshed)
    cv2.waitKey(1)
    return
```

C.6 Sobel Detection

```
def ContourDetection_Sobel():
    global center, tip
    ##### Find the Edges of the img, then put all of the edge pts into a vector
    try:
        sobelx = cv2.Sobel(eroded, cv2.CV_64F, 1, 0, ksize=3)
        sobely = cv2.Sobel(eroded, cv2.CV_64F, 0, 1, ksize=3)
        sobelMAG=np.uint8(np.sqrt(np.add(np.multiply(sobelx, sobelx),
                                                np.multiply(sobely, sobely))))
        cv2.imshow("Sobel_Edge_Detector_Mag", sobelMAG)
        silhouette = np.array(np.where(sobelMAG > 100))
        # Calculate centroid (avg pt from edge) in y,x
        silhouette_length = silhouette.shape[1]
        sum_x = np.sum(silhouette[1, :])
        sum_y = np.sum(silhouette[0, :])
        center = [sum_x/silhouette_length, sum_y/silhouette_length]
        # Defines the tip coords as the furthest from the center.
        # This calc is only really possible because of shape of ship
        dif = np.array([np.subtract(silhouette[0, :], center[1]),
                        np.subtract(silhouette[1, :], center[0])])
        normal = np.add(np.multiply(dif[0, :], dif[0, :]),
                        np.multiply(dif[1, :], dif[1, :]))
        tip = silhouette[:, np.argmax(normal)]
    except:
        print("Sobel_Line_Detection_Failed")
        ContourDetection_findContours() #Try this instead
    return
```

C.7 FindContour() Detection

```
def ContourDetection_findContours():
    global center, tip
    im_dummy, contours, hierarchy = cv2.findContours(eroded, \
                                                    cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)

    try:
        cnt=sorted(contours, key=cv2.contourArea, reverse=True)[0]#contours[
            np.argmax(len(contours[0][:]))]
        # Moment and centroid definition
        M = cv2.moments(cnt)
        center = [int(M['m10'] / M['m00']), int(M['m01'] / M['m00'])] # y, x
        # Finds the avg of the contour, defines tip as furthest from the avg
        .
        # This calc is only usefull because of shape of ship
        dif = np.subtract(cnt[:,], center[:])
        normal = np.sqrt(np.add(np.multiply(dif[:, 0], dif[:, 0]),
                                           np.multiply(dif[:, 1], dif[:, 1])))
        tip = [cnt[np.argmax(normal)][0,1], cnt[np.argmax(normal)][0,0]]

    except:
        print "Wow_something_broke_the_contours"

    return
```

C.8 Position Calculator

```
def Position_Calc():
    global position, theta, theta_Relative, ship_length, scale, velocity
    # Angle calculation based on slopes of line
    dy = -center[0] + tip[1]
    dx = -center[1] + tip[0]
    theta = math.degrees(math.atan2(dy, dx))
    # Define relative coordinates for command line
    theta_Relative = (theta_Target - theta)
    if theta_Relative > 180:
        theta_Relative = theta_Relative - 360
    elif theta_Relative < -180:
        theta_Relative = theta_Relative + 360

    # Use scale of ship to determine actual distances
    ship_length=np.linalg.norm([dx,dy]) # Pixels long
    #print(ship_length)
    # using known length to determine m/pix scale
    scale = real_length / ship_length
    # Scales relative position (x,y)
    position = np.multiply(np.dot(Rotation,
                                   [center[0] - Target[0], center[1] - Target[1]]), scale)
    # Define the velocity vector of the ship
    velocity = np.divide([theta_Relative - theta_old,
                          position[1] - position_old[1], position[0] - position_old[0]], dT)

    return
```


C.9 Command Calculator

```
def Command_Calc():
    global u, v, Command, u_old, theta_old, position_old
    # Calculate command coefficients
    for ii in range(0, 3):
        a0[ii] = P[ii] * dT + 2
        c1[ii] = (dT * K[ii] * Z[ii] + 2.0 * K[ii]) / a0[ii]
        c2[ii] = (dT * K[ii] * Z[ii] - 2.0 * K[ii]) / a0[ii]
        c3[ii] = (dT * P[ii] - 1.0) / a0[ii]
    # If theta not resolved yet
    v = [0, 0, 0, 0, 0, 0]
    if abs(theta_Relative) > theta_threshold:
        u[0] = c1[0] * theta_Relative + c2[0] * theta_old - c3[0] * u_old[0]
        if u[0] >= 0:
            v[0] = m*u[0] + b
            v[3] = v[0]
        elif u[0] <= 0:
            v[1] = -m*u[0] + b
            v[2] = v[1]
    else: #abs(position[0]) > x_threshold or abs(position[1]) > y_threshold:
        u[0] = c1[0] * theta_Relative + c2[0] * theta_old - c3[0] * u_old[0]
        # Position commands use scaled position to get values in meters
        u[1] = c1[1] * position[0] + c2[1] * position[0] - c3[1] * u_old[1]
        u[2] = c1[2] * position[1] + c2[2] * position[1] - c3[2] * u_old[2]
        if u[0] <= 0:
            v[0] = -m*u[0]
            v[3] = v[0]
        elif u[0] > 0:
            v[1] = m*u[0]
            v[2] = v[1]
        if u[1] < 0:
            v[0] = -m*u[1] + b + v[0]
            v[2] = v[0] + v[2]
        elif u[1] > 0:
            v[1] = m*u[0] + b + v[1]
            v[3] = v[1] + v[3]
        if u[2] < 0:
            v[4] = -m*u[2] + b
        elif u[2] > 0:
            v[5] = m*u[2] + b
    for ii in range(0, 6):
        if v[ii] > 255 or v[ii] < 0:
            v[ii] = 255
        elif v[ii] > 0 and v[ii] < b:
            v[ii] = b
    # Writes to the serial port the command line of motor speeds
    Command = 'A' + str(int(v[0])) + 'B' + str(int(v[1])) +
              'C' + str(int(v[2])) + 'D' + str(int(v[3])) +
              'E' + str(int(v[4])) + 'F' + str(int(v[5])) + 'Z'
    ser.write(Command)
    print ('Send to Shuttle:..' + Command)
    u_old = u
    position_old = position # Store position for next calculation
    theta_old = theta_Relative
    return
```

C.10 Flame Projection

```
def Flame_Projection():
# Flame Projection
# Create array of anchor points as they relate to a downward facing shuttle
anchor_points = np.divide(Anchors,(standard_length/ship_length))

# Rotate image to figure out layout
Ship_Rotation = cv2.getRotationMatrix2D((center[0],center[1]),-theta,1)
rotated = cv2.warpAffine(image_focused,Ship_Rotation,(int(cols*2/portion),int(rows
    *2/portion)))
try:
if v[5] > 0: # Front
    Scaled_Flame = cv2.resize(Flame_front, dsize = (0,0), fx = v[5]/(jet_scale
        *255.0), fy = v[5]/(jet_scale*255.0))
    Flame_size = np.shape(Scaled_Flame)
    alpha_s = Scaled_Flame[:, :, 3] / 255.0
    alpha_l = 1.0 - alpha_s
    TL = np.array([anchor_points[5][0]+center[0]-Flame_size[1]/2 + offset,
        anchor_points[5][1]+center[1]]).astype('int16')
    BR = np.add(TL, [Flame_size[1] + offset, Flame_size[0]]).astype('int16')
    for c in range(0, 3):
        rotated[TL[1]:BR[1], TL[0]:BR[0], c] = (alpha_s * Scaled_Flame[:, :,
            c] + alpha_l * rotated[TL[1]:BR[1], TL[0]:BR[0], c])
if v[4] > 0: # rear
    Scaled_Flame = cv2.resize(Flame_rear, dsize = (0,0), fx = v[4]/(jet_scale
        *255.0), fy = v[4]/(jet_scale*255.0))
    Flame_size = np.shape(Scaled_Flame)
    alpha_s = Scaled_Flame[:, :, 3] / 255.0
    alpha_l = 1.0 - alpha_s
    TL = np.array([anchor_points[4][0]+center[0]-Flame_size[1]/2 + offset,
        anchor_points[4][1]+center[1]-Flame_size[0]]).
        astype('int16')
    BR = np.add(TL, [Flame_size[1] + offset, Flame_size[0]]).astype('int16')
    for c in range(0, 3):
        rotated[TL[1]:BR[1], TL[0]:BR[0], c] = (alpha_s * Scaled_Flame[:, :,
            c] + alpha_l * rotated[TL[1]:BR[1], TL[0]:BR[0], c])
if v[0] > 0: # Right Front
    Scaled_Flame = cv2.resize(Flame_right, dsize = (0,0), fx = v[0]/(jet_scale
        *255.0), fy = v[0]/(jet_scale*255.0))
    Flame_size = np.shape(Scaled_Flame)
    alpha_s = Scaled_Flame[:, :, 3] / 255.0
    alpha_l = 1.0 - alpha_s
    TL = np.array([anchor_points[0][0]+center[0]-Flame_size[1] + offset,
        anchor_points[0][1]+center[1]-Flame_size[0]/2]).
        astype('int16')
    BR = np.add(TL, [Flame_size[1] + offset, Flame_size[0]]).astype('int16')
    for c in range(0, 3):
        rotated[TL[1]:BR[1], TL[0]:BR[0], c] = (alpha_s * Scaled_Flame[:, :,
            c] + alpha_l * rotated[TL[1]:BR[1], TL[0]:BR[0], c])
if v[2] > 0: # Right back
    Scaled_Flame = cv2.resize(Flame_right, dsize = (0,0), fx = v[2]/(jet_scale
        *255.0), fy = v[2]/(jet_scale*255.0))
    Flame_size = np.shape(Scaled_Flame)
    alpha_s = Scaled_Flame[:, :, 3] / 255.0
    alpha_l = 1.0 - alpha_s
    TL = np.array([anchor_points[2][0]+center[0]-Flame_size[1] + offset,
```

```

                                anchor_points[2][1]+center[1]-Flame_size[0]/2)) .
                                astype('int16')
BR = np.add(TL, [Flame_size[1] + offset, Flame_size[0]]).astype('int16')
for c in range(0, 3):
    rotated[TL[1]:BR[1], TL[0]:BR[0], c] = (alpha_s * Scaled_Flame[:, :,
        c] + alpha_l * rotated[TL[1]:BR[1], TL[0]:BR[0], c])
if v[1] > 0: # left Front
    Scaled_Flame = cv2.resize(Flame_left, dsize = (0,0), fx = v[1]/(jet_scale
        *255.0), fy = v[1]/(jet_scale*255.0))
    Flame_size = np.shape(Scaled_Flame)
    alpha_s = Scaled_Flame[:, :, 3] / 255.0
    alpha_l = 1.0 - alpha_s
    TL = np.array([anchor_points[1][0]+center[0] + offset,
        anchor_points[1][1]+center[1]-Flame_size[0]/2)) .
        astype('int16')
    BR = np.add(TL, [Flame_size[1] + offset, Flame_size[0]]).astype('int16')
    for c in range(0, 3):
        rotated[TL[1]:BR[1], TL[0]:BR[0], c] = (alpha_s * Scaled_Flame[:, :,
            c] + alpha_l * rotated[TL[1]:BR[1], TL[0]:BR[0], c])
if v[3] > 0: # left Front
    Scaled_Flame = cv2.resize(Flame_left, dsize = (0,0), fx = v[3]/(jet_scale
        *255.0), fy = v[3]/(jet_scale*255.0))
    Flame_size = np.shape(Scaled_Flame)
    alpha_s = Scaled_Flame[:, :, 3] / 255.0
    alpha_l = 1.0 - alpha_s
    TL = np.array([anchor_points[3][0]+center[0] + offset,
        anchor_points[3][1]+center[1]-Flame_size[0]/2)) .
        astype('int16')
    BR = np.add(TL, [Flame_size[1] + offset, Flame_size[0]]).astype('int16')
    for c in range(0, 3):
        rotated[TL[1]:BR[1], TL[0]:BR[0], c] = (alpha_s * Scaled_Flame[:, :,
            c] + alpha_l * rotated[TL[1]:BR[1], TL[0]:BR[0], c])
except:
    print('One_of_these_flames_got_Busted')
Ship_Rotation = cv2.getRotationMatrix2D((center[0], center[1]), theta+1.25, 1)
unrotated = cv2.warpAffine(rotated, Ship_Rotation, (int(cols*2/portion), int(rows*2/
    portion)))
cv2.imshow('projected', rotated)
cv2.imshow('projected_and_returned_to_OG_orientation', unrotated)
return

```

C.11 Position Overlay

```
def Position_Overlay():
    global img, Trail
    # Shows the centroid, tip, line between (Y,X)
    cv2.line(img, (center[0], center[1]), (tip[1], tip[0]), 255, 2)
    cv2.circle(img, (center[0], center[1]), 2, 100, 1)
    cv2.circle(img, (tip[1], tip[0]), 2, 0, 1)
    # Draw Target location
    cv2.circle(img, (Target[0], Target[1]), 2, 200, 2)
    cv2.line(img, (Target[0], Target[1]), (tip_Target[0], tip_Target[1]), 50, 2)
    # Draw trail
    Trail.appendleft((center[0], center[1]))
    for jj in xrange(1, len(Trail)):
        if Trail[jj-1] is None or Trail[jj] is None:
            continue # Skips iteration since no line can be drawn
        thickness = int(np.sqrt(Buffer_Length)/ float(jj+1))*2
        cv2.line(img, Trail[jj-1], Trail[jj], 200, thickness)

    # Display images for testing purposes.
    cv2.imshow('Centroid', img)
    # Display info
    print ('Orientation:', theta_Relative, "Postion:", position, 'dT', dT)
    return
```

C.12 Save Data

```
def Save_Data():
    global ii, Data
    # Save data as (rel X, rel y, rel Theta, dT, abs x, abs y, abs theta)
    Data[ii] = [position[1], position[0], theta_Relative, dT,
                u[1], u[2], u[0], v[0], v[1], v[2], v[3], v[4], v[5],
                center[1], center[0], int(math.degrees(theta))]
    # Write an image of the tracked object to a folder
    #IMGfname = "/home/pi/Desktop/AutoPilot/CV/ImageStore/Control_Test1_img%s.
    png" % i
    #cv2.imwrite(IMGfname, img)
    return
```

D Data Processing

```

%Import data as (relative X, relative y, relaive Theta, dT,...
% u's, v's abs center x, abs center y, abs theta)
Data=csvread( 'Data_Collection_Controlled_1.csv' );
n=size(Data,1);
Position=Data(:,1:3);
t=zeros(n,1); Velocity=zeros(n,3); Acceleration=zeros(n,3);
for i=2:n
    t(i)=t(i-1)+Data(i-1,4);
    Velocity(i,:)=(Position(i,:)-Position(i-1,:))/Data(i-1,4);
    Acceleration(i,:)=(Velocity(i,:)-Velocity(i-1,:))/Data(i-1,4);
end
Commands = Data(:,5:7);
Fan_Speeds = Data(:,8:13);
figure(1);subplot(3,1,1); hold on; ylabel('Position_(Pixels)')
plot(t,Position(:,1),t,Position(:,2),t,Position(:,3));
subplot(3,1,2); hold on; ylabel('Velocity')
plot(t,Velocity(:,1),t,Velocity(:,2),t,Velocity(:,3));
subplot(3,1,3); hold on; ylabel('Acceleration')
plot(t,Acceleration(:,1),t,Acceleration(:,2),t,Acceleration(:,3));
legend('X','Y','Theta'); figure(2);subplot(2,1,1); hold on;
plot(t,Commands(:,1),t,Commands(:,2),t,Commands(:,3));
legend('Ux','Uy','Utheta'); subplot(2,1,2);hold on;
plot(t,Fan_Speeds(:,1),t,Fan_Speeds(:,2),t,Fan_Speeds(:,3),...
    t,Fan_Speeds(:,4),t,Fan_Speeds(:,5),t,Fan_Speeds(:,6));
% Shuttle parameter calcs
A=.0696; %m^2 - surf area, calculated on SW
m=.85; %kg - measured
% After T>5, the force cuts out, so the acceleration from there will
% inform the damping value b. Damping force is proportional to speed, as
% in this eqn: m*Ax=b*Vx
Bcalcs=zeros(n,3);
for i=1:3
    Bcalcs(:,i)=m*Acceleration(:,i)./Velocity(:,i);
end
bAvg=[sum(Bcalcs(:,1))/n,sum(Bcalcs(:,2))/n,sum(Bcalcs(:,3))/n]
% Knowing b, we look at the accel in T<5, which should be relatively
% constant, equalling Ffan+Fdrag

```